

XRan: Explainable deep learning-based ransomware detection using dynamic analysis

Sibel Gulmez^{a,*}, Arzu Gorgulu Kakisim^b, Ibrahim Sogukpinar^a

^a Gebze Technical University, Computer Engineering Department, Gebze, Kocaeli, Turkey

^b Istanbul Medeniyet University, Computer Engineering Department, Uskudar, Istanbul, Turkey

ARTICLE INFO

Keywords:

Ransomware detection
Dynamic analysis
Deep learning
XAI
API calls
DLLs
Mutual exclusions

ABSTRACT

Recently, the frequency and complexity of ransomware attacks have been increasing steadily, posing significant threats to individuals and organizations alike. While traditional signature-based antiransomware systems are effective in the detection of known threats, they struggle to identify new ransomware samples. To address this limitation, many researchers have focused on analyzing the behavior and actions of executables. During this dynamic analysis process, various dynamic-based features emerge, offering different perspectives on the executable's behavior, including Application Program Interface (API) call sequences, dynamic link libraries (DLLs), and mutual exclusions. Existing methods mostly perform machine or deep learning models for feature engineering and detection. These methods usually perform learning according to a single perspective or by combining data from different perspectives into the frequency domain. In this case, they may ignore the information from the other aspects or the sequence relationship between the features. In addition, learning models used in these solutions are mostly incomprehensible to humans, which could be an obstacle in terms of having an insight through the model's mentality and also ransomware's way of work. In this study, we provide XRan (eXplainable deep learning-based RANsomware detection using dynamic analysis), an Explainable Artificial Intelligence (XAI) supported ransomware detection system that combines different dynamic analysis-based sequences, each representing a different view of the executable, in order to enrich the feature space. XRan employs a Convolutional Neural Network (CNN) architecture to detect ransomware and two XAI models as Interpretable Model-Agnostic Explanations (LIME), and SHapley Additive exPlanations (SHAP) to provide local and global explanations for detection. Experimental results demonstrate that XRan provides up to 99.4% True Positive Rate (TPR), and outperforms the state-of-the-art methods.

1. Introduction

With its diversity and impact, malware has long been one of the most dangerous cyber threats. Attackers use malware for various purposes such as data leakage, disruption or destruction of computer systems, reputational damage, financial gain, and identity theft. Based on the intent and the characteristics of the attack, malware can be classified into several families. Virus, worm, trojan, adware, spyware, and ransomware are some of the most well-known and common malware families (Conti et al., 2022; Yang et al., 2023). In recent times, there has been a notable surge in both the frequency and complexity of ransomware attacks, presenting a growing menace to individuals and organizations. These malicious activities can cause significant financial loss by encrypting victims' files or devices and demanding payment for

restoring access, or damaging victims' reputations by disseminating victim information and data.

Ransomware has historically been categorized into three sub-families: crypto, locker, and scareware. Recently, new ransomware variants are also becoming popular, including Doxware or leakware, and ransomware as a service (RaaS). Without interfering with basic computer functions, crypto-ransomware encrypts files and data within a system, making content inaccessible without a decryption key. If encryption techniques available in crypto-ransomware are applied correctly, the consequences of this attack are almost impossible to reverse (Beaman et al., 2021). Locker ransomware primarily aims to block computer functions. The ransom demand is delivered to the victims, usually via a lock screen with a countdown clock to quickly activate the victims. Unlike crypto-ransomware, the consequences of a Locker ransomware

* Corresponding author.

E-mail addresses: sgulmez2018@gtu.edu.tr (S. Gulmez), arzu.kakisim@medeniyet.edu.tr (A. Gorgulu Kakisim), ispinar@gtu.edu.tr (I. Sogukpinar).

<https://doi.org/10.1016/j.cose.2024.103703>

Received 18 August 2023; Received in revised form 15 November 2023; Accepted 2 January 2024

Available online 8 January 2024

0167-4048/© 2024 Elsevier Ltd. All rights reserved.

attack can usually be avoided by restarting the computer in safe mode and running a virus scanner (Adamu and Awan, 2019). Scareware is a type of ransomware that aims to convince users that they need to download certain malicious software, usually by claiming to have detected a virus or other problem on their computer, while actually doing no harm to the victim's computer. This ransomware often fills the screen with pop-up alerts, causing users to panic and download software that forces them to pay to fix the problem.

Attackers often demand ransom to be paid via an untraceable currency, e.g. bitcoin, so that their traces are not found. As expected, this situation makes it difficult to identify the attacker. Besides that, the financial gain also motivates the attackers into carrying out ransomware attacks. As a consequence, ransomware has made its mark with its increasing volume and influence in the last decade. A great number of individuals and organizations from various industries such as education, healthcare, and finance became the target of ransomware threats. Billions of dollars were paid as ransom and/or spent for the data recovery process. Although it is observed that the volume of ransomware attacks decreased in 2022 (SonicWall, 2023), the percentage of the organizations that were infected by ransomware had increased compared to 2021 (Statista, 2023). Besides, the number of reported ransomware drastically increased in 2023. In the first 7 months of 2023, the number of reported ransomware has outdistanced the number in the same month of the last 4 years, except in April. In May 2023, the difference got even more apparent by doubling the numbers from the last 4 years. Moreover, in July 2023, the amount of ransom payout increased by 126% percent from the first quarter of 2023 (BlackFog, 2023). The statistics put the importance of ransomware detection studies into perspective.

In the last 8 years, many studies have been conducted on the detection of ransomware. Most of these studies are based on dynamic analysis, which is a malware analysis technique that necessitates running the software on a sandbox (Chen et al., 2017; Qin et al., 2020; Karbab et al., 2023). The main reason for this is that the attackers usually employ code obfuscation and/or encryption techniques in an attempt to bypass static analysis, which is also known as code analysis with its structure based on the analysis of the source code of an executable. A classification task follows the analysis process with the purpose of automatizing the detection of ransomware based on the information obtained by the analyses (Sheen and Yadav, 2018; Moreira et al., 2023). The classification, in terms of detecting the ransomware, is a binary classification that labels software as ransomware or benign. As in many malware detection studies, deep learning and/or machine learning algorithms are widely used for this purpose (Hasan and Rahman, 2017; Alhawi et al., 2018; Manavi and Hamzeh, 2022a). Recent scholarly research has classified ransomware detection methodologies into two distinct categories. The initial category focuses on the utilization of real-time monitors to analyze real-time data, thereby enabling the detection of any abnormal alterations in system behavior. Investigations falling within the second category employ both dynamic and static analyses on representative program samples to ascertain the presence of ransomware. The process of detection is implemented through the application of machine learning and deep learning approaches, utilizing attributes encompassing API calls, system calls, opcodes, raw bytes, and additional features derived from these analyses. However, prevailing methodologies often engage in the learning process from a singular feature such as API calls or system calls (Sgandurra et al., 2016; Qin et al., 2020; Moreira et al., 2023) or by combining different attributes obtained from various perspectives into the frequency domain (Shaukat and Ribeiro, 2018; Hwang et al., 2020). In this case, they may ignore the information from the other perspectives or the sequence relationship between the features.

Moreover, machine/deep learning algorithms generally offer a black-box, presenting no information about the way it works. Understanding the reasons behind a classification model's decisions could be quite useful in terms of understanding the manner of work of the object

to be detected (Keshk et al., 2023; Suhail et al., 2023). For this purpose, XAI algorithms have been developed. These algorithms provide intelligible information about the model's way of working and clear the way between human and artificial intelligence. As a matter of course, these models have started to be used in a very wide field of work together with artificial intelligence models. Especially since malware detection is a critical task, understanding the characteristics or patterns that cause a particular file or activity to be classified as malicious is important for security professionals. Because XAI models highlight which features or characteristics of data are most effective in making predictions, security analysts can use this information to strengthen defenses against certain types of attacks. Malicious actors may attempt to manipulate or evade detection systems. In this case, XAI models can help identify situations where the model may have made the wrong decisions. In this way, it helps security experts draw a roadmap to improve the model, reduce false positives and false negatives, and increase overall detection accuracy. Additionally, the attributes revealed by XAI models can be used to provide informative feedback for a user using the security system. XAI results can be used to explain the reasons why software classified as malware was quarantined. For example, disclosing certain information regarding an objectionable file to which access is requested or unusual use of an API call serves to establish a foundation of trust between the user and the software. In this direction, recently, newly developed artificial intelligence-based malware detection systems have started to be integrated with XAI models (Hsupeng et al., 2022; Alani et al., 2023).

In this study, we propose a novel deep learning-based ransomware detection method, called XRun, that leverages dynamic analysis and is further augmented with XAI models. Firstly, the proposed method extracts API call sequences, Dynamic Link Library (DLL) sequences, and Mutual Exclusion (Mutex) sequences from both malicious and benign samples. These sequences are derived through a dynamic analysis process that includes running programs in a virtual environment and monitoring their behavior. Afterward, for each executable, XRun combines the three distinct sequences and treats them as input data for the two-layered CNN model. The CNN model processes the combined sequences and performs the detection of ransomware. By utilizing dynamic analysis and XAI models, our method aims to enhance the accuracy and interpretability of ransomware detection, offering a more effective and understandable approach to identifying potential threats in executable files.

The main contributions of our study are presented as follows:

- We provide a robust ransomware detection method based on dynamic analysis. The method combines multiple different sequences, each representing a different view of the executable, to enrich the feature space. The expanded feature space offers a more detailed and comprehensive set of features, providing in-depth and nuanced insights. XRun learns the embeddings through a joint space composed of the network structure and attributes. By creating concatenated sequences, XRun learns a joint space, not only according to the order relationship between the API call attributes, for example, but also capturing the correlation between API call and DLL, API call and mutex, and DLL and mutex attributes. This enriched joint space empowers our CNN model with more granular intelligence, enabling it to make better-informed and more precise decisions.
- Our proposed method exhibits the capability not only to detect ransomware but also to distinguish it from general malware. The method is able to capture ransomware from general malware with a high degree of accuracy.
- XRun empowers the detection method using LIME and SHAP models to provide explainability. Through XAI models, we aim to shed light on the decision-making process of our proposed model and provide insight into the factors that will enable them to predict. We provide a detailed analysis of the patterns, behaviors, and consequences that distinguish ransomware from benign software, as well as distinguish it from other types of malware. Although there

are studies combining XAI and malware detection methods, our method would be a pioneering study using XAI for ransomware detection to the best of our knowledge.

- Experiments are conducted on four different ransomware datasets to evaluate the performance of the proposed method. Experimental results show that the XRank significantly achieves a higher True Positive Rate (TPR) than the state-of-the-art methods.

The rest of the paper is organized as follows. In Section 2, a literature review on the detection of ransomware is provided and also the usage of XAI models for malware detection. Section 3 describes the proposed ransomware detection framework. The experimental setup and test results are given in Section 4. Section 5 concludes the paper with remarks and future work ideas.

2. Related work

In this section, firstly we refer to the prominent studies on ransomware detection. We also summarize these studies in Table 1. Secondly, we review academic research on the usage of XAI for malware detection.

2.1. Ransomware detection

In recent years, researchers have shown significant interest in addressing the pressing issue of safeguarding computer systems against ransomware threats. There have been notable efforts to propose detection systems specifically tailored to identifying ransomware samples. Recent research in ransomware detection has been categorized into two distinct groups. The first category centers on detection approaches that utilize real-time monitors to analyze instant data, enabling the identification of any abnormal changes in system behavior (Karbab et al., 2023). Continella et al. (2016) introduced ShieldFS, a novel approach for ransomware detection. ShieldFS utilizes low-level filesystem monitoring to construct a baseline or *normal* profile over time. Any significant deviation from this established normal profile is indicative of potential malicious activity. Kharaz et al. (2016) introduced a system called UNVEIL, which is designed to detect ransomware-like activities, including file encryption and desktop locking. UNVEIL leverages I/O request monitoring and employs Shannon entropy to differentiate encrypted files from regular ones. The research conducted by Morató et al. presented REDFISH (Morató et al., 2018). It is a ransomware detection system that is capable of identifying ransomware activity by monitoring network traffic. Almashhadani et al. (2019) proposed a multi-classifier network-based ransomware detection method that aims to extract potential behavioral features through several network activities such as TCP, HTTP, and DNS. In the studies by Lee et al. (2019), entropy analysis is employed as a method to detect infected files. By evaluating the entropy levels of files, these approaches can identify potential ransomware infections based on deviations from expected entropy patterns. McIntosh et al. proposed a user-centric access control framework (McIntosh et al., 2021). This framework is designed to assess file access requests and make determinations based on indicators collected from the operating system.

Studies in the second category employ dynamic and/or static analysis on program samples to identify ransomware. Attributes such as API calls, system calls, opcodes, raw bytes, and other features are extracted through static or dynamic analysis. Detection is carried out with machine learning and deep learning approaches over these attributes. HelDroid (Andronio et al., 2015) was the very first ransomware detection tool that was developed in 2015. It is based on the detection of the following three: threatening text, encryption, and locking. The detection of the threatening text performs both static and dynamic analysis while the rest of the framework is being carried out through static analysis only. Sgandurra et al. (2016) proposed a dynamic analysis-based ransomware detection method. Their method uses some dynamic features such as

Windows API calls, registry key operations, file system operations, and directory operations. It applies a mutual information-based feature selection mechanism to capture the most relevant features. To detect ransomware samples, it uses Regularized Logistic Regression. Sheen and Yadav (2018) presented an API call-based detection system. They extracted frequently observed API calls and performed different machine learning algorithms such as Decision Tree and Naive Bayes for ransomware detection. They also provided a feature generation phase using SMOTE and applied various machine learning methods for ransomware detection. Shaukat and Ribeiro (2018) presented a layered defense system for cryptographic ransomware. Their method consists of a strong trap layer for early detection and a machine learning layer for zero-day sample detection. Hwang et al. (2020) proposed a two-stage mixed ransomware detection model that consists of the Markov model and the Random Forest (RF) algorithm. They first focused on discovering the Windows API call characteristics of ransomware, and then merging the information with remaining data such as registry keys, file extensions, etc. Jethva et al. (2020) developed a multi-layer ransomware detection technique. The first layer of the proposed method benefits from dynamic features while the second layer makes use of entropy and file signature. Manavi and Hamzeh (2022a) proposed a graph embedding-based ransomware detection method that utilizes the headers of executable files. Their method creates a graph, generates a feature space using the Power Iteration method, and classifies ransomware samples using the Random Forest classifier. Manavi and Hamzeh (2022b) presented a deep learning-based detection system that converts PE header information to images and implements CNN to detect ransomware samples. Similarly, Moreira et al. (2023) used static analysis for detecting ransomware samples by converting PE header information into images, and applied Xception Convolutional Neural Network model.

2.2. XAI-empowered malware detection

AI models consistently exhibit remarkable proficiency in the realm of feature engineering, while concurrently tackling progressively intricate datasets. As innovative architectures are harnessed, their prowess in addressing complex problems undergoes further evolution, thereby giving rise to a substantial upsurge in the proliferation of applications grounded in AI. Despite the apparent efficacy of AI algorithms in generating outcomes and forecasts, their lack of transparency introduces a challenge in comprehending their internal operational mechanisms. This issue is particularly pronounced within fields such as cybersecurity, where delegating crucial decisions to a system incapable of self-explanation entail obvious hazards. If these systems fail to grasp and derive insights from cybersecurity incidents, they devolve into a black-box with inherent risks. Consequently, there arises a demand for statistical AI algorithms that facilitate the quantification of such uncertainties. Herein, explainable AI (XAI) assumes a pivotal role by conferring interpretability onto the outcomes engendered by AI-powered statistical models. This affords researchers and experts the capacity to fathom causal rationales and primary empirical evidence underpinning these outcomes. For this reason, some researchers have recently focused on integrating their newly developed AI-based malware detection systems with explainable AI (XAI) models.

A notable contribution in this domain is the work proposed by Arp et al. (2014). To identify malicious activities in unknown applications, their system named Drebin represented each mobile application with eight distinct feature sets from varied sources, including API calls, requested permissions, and content providers. Drebin utilized an SVM-based malware detection model to classify the applications as malware or benign. In terms of explanation, Drebin reported the most influential features having the highest absolute weights assigned by the classifier. In the realm of Android malware detection, Fan et al. (2020) used five distinct local and model-agnostic explanation methods including LIME (Ribeiro et al., 2016), Anchor (Ribeiro et al., 2018), LORE (Guidotti et al., 2018), SHAP (Lundberg and Lee, 2017), and LEMNA

Table 1

Different methods for ransomware detection in the literature.

Paper	Feature	Analysis Technique	Detection Method	Dataset Size	Dataset Source	TPR	FPR	ACC
Sgandurra et al. (2016)	API calls, registry keys, file system, directory operations, etc.	Dynamic Analysis	Mutual Information based feature selection + Regularized LR	582 Ransomware 942 Benign	VirusShare (VirusShare, 2016) Software.Informer (Software Informer, 2016)	0.9634	0.0161	N/A
Hasan and Rahman (2017)	API calls, Registry key, process, network operations, etc.	Hybrid Analysis	Mutual Information based feature selection + SVM	360 Ransomware 460 Benign	VirusShare (VirusShare, 2016) N/A	0.96	N/A	0.961
Chen et al. (2017)	API call flow graph	Dynamic Analysis	Feature Selection with Gain Ratio + Simple Logistic	83 Ransomware 85 Benign	VirusShare (VirusShare, 2016) Software.Informer (Software Informer, 2016)	0.976	0.012	N/A
Sheen and Yadav (2018)	API calls	Static Analysis	SMOTE + Random Forest	16,243 Ransomware 3,620 Benign	VirusShare (VirusShare, 2016) N/A	0.9845	0.015	N/A
Shaukat and Ribeiro (2018)	PE header, embedded resources, embedded strings, etc.	Hybrid Analysis	Entropy + Gradient Tree Boosting	574 Ransomware 442 Benign	VirusShare (VirusShare, 2016) N/A	0.9825	0.056	N/A
Alhawi et al. (2018)	Network traffic	Dynamic Analysis	Decision Tree	210 Ransomware 264 Benign	VirusTotal (VirusTotal, 2018) VirusTotal (VirusTotal, 2018)	0.9710	0.0160	N/A
Hwang et al. (2020)	API calls, Registry key, file system, directory operations, etc.	Dynamic Analysis	Markov Model + Random Forest	1,176 Ransomware 1,160 Benign	VirusShare (VirusShare, 2016) Softonic.Com (Softonic, 2020)	N/A	0.0483	0.9728
Qin et al. (2020)	API call sequences	Dynamic Analysis	TextCNN	1,000 Ransomware 1,000 Benign	Sangfor Technologies (Sangfor Technologies, 2020) Sangfor Technologies (Sangfor Technologies, 2020)	N/A	N/A	0.959
Manavi and Hamzeh (2022a)	PE header bytes	Static Analysis	Graph embedding + Random Forest	6,000 Ransomware 6,000 Benign	VirusShare (VirusShare, 2016) Windows System Files, SnapFiles.Com (SnapFiles, 2022), and FreewareFiles.Com (FreewareFiles, 2022)	N/A	N/A	0.9495
Manavi and Hamzeh (2022b)	PE header bytes	Static Analysis	CNN	1,000 Ransomware 1,000 Benign	VirusShare (VirusShare, 2016) SnapFiles.Com (SnapFiles, 2022), FreewareFiles.Com (FreewareFiles, 2022), and PortableApps.Com	N/A	N/A	0.9333
Moreira et al. (2023)	PE header bytes based images	Static Analysis	xception CNN	2,023 Ransomware 2,134 Benign	VirusShare (VirusShare, 2016), and Hybrid-Analysis (Free Automated Malware Analysis Service, 2020) N/A	N/A	N/A	0.9373
Jethva et al. (2020)	API calls, DLL, Registry key operations, strings, entropy, etc.	Hybrid Analysis	SVM, Random Forest, Logistic Regression	666 Ransomware 103 Benign	ISOT (Jethva et al., 2020) Software.Informer (Software Informer, 2016)	1.000	0.133	0.987

(Guo et al., 2018). Similarly, the authors of (Kinkead et al., 2021) utilized LIME to identify key patterns highlighted by a CNN within an Android application's opcode sequence. This method aids in malware detection by pinpointing critical regions. Feichtner and Gruber (2020) proposed a Convolutional Neural Network (CNN) that identifies correlations between segments of description text and the permission groups requested by an application. LIME is incorporated to calculate the significance score of each word in the output, which is then visualized as a heatmap. Wu et al. (2021) employed a multi-layer perceptron (MLP) with an attention mechanism for malware classification. They reported the explanations for each prediction based on the features that contribute most significantly to the prediction. These features are determined based on the attention weights. This approach adds to their model the ability to provide local explanations for the predictions produced by the model. Pan et al. (2022) proposed a hardware-assisted malware detection framework and presented an Explainable Machine Learning algorithm based on regression. They utilize a Decision Tree (DT) or Linear Regression (LR) to interpret the final outcome. Hsupeng et al. (2022) presented an explainable flow-data classification model for identifying hacker attacks by preprocessing flow data from packets. For explainability, they employed SHAP to explore the relationship

between cyberattacks and network flow features. Alani et al. (2023) developed XMal which is a lightweight memory-based malware detector. XMal uses only five features extracted from memory dumps and an extreme gradient boost for detection. The global explanations of XMal are provided using SHAP.

3. Proposed method: XRan

In this work, we propose XRan, a novel ransomware detection method based on the combination of multiple dynamic features. XRan employs a well-known deep learning algorithm, CNN, supported by XAI models SHAP and LIME. The main purpose of the study is to detect ransomware at the highest possible rate and to explain the features that are decisive in the decision-making mechanism of the model. XRan's workflow is visualized in Fig. 1. XRan was trained on both ransomware/benign and ransomware/malware datasets, creating two separate models using two different training sets. In this way, two different models are used in parallel to determine whether a new sample is ransomware. Here, it is aimed to more clearly reveal the characteristics that distinguish ransomware from other examples by observing the explainable results offered by the XRan model on both benign and malware sam-

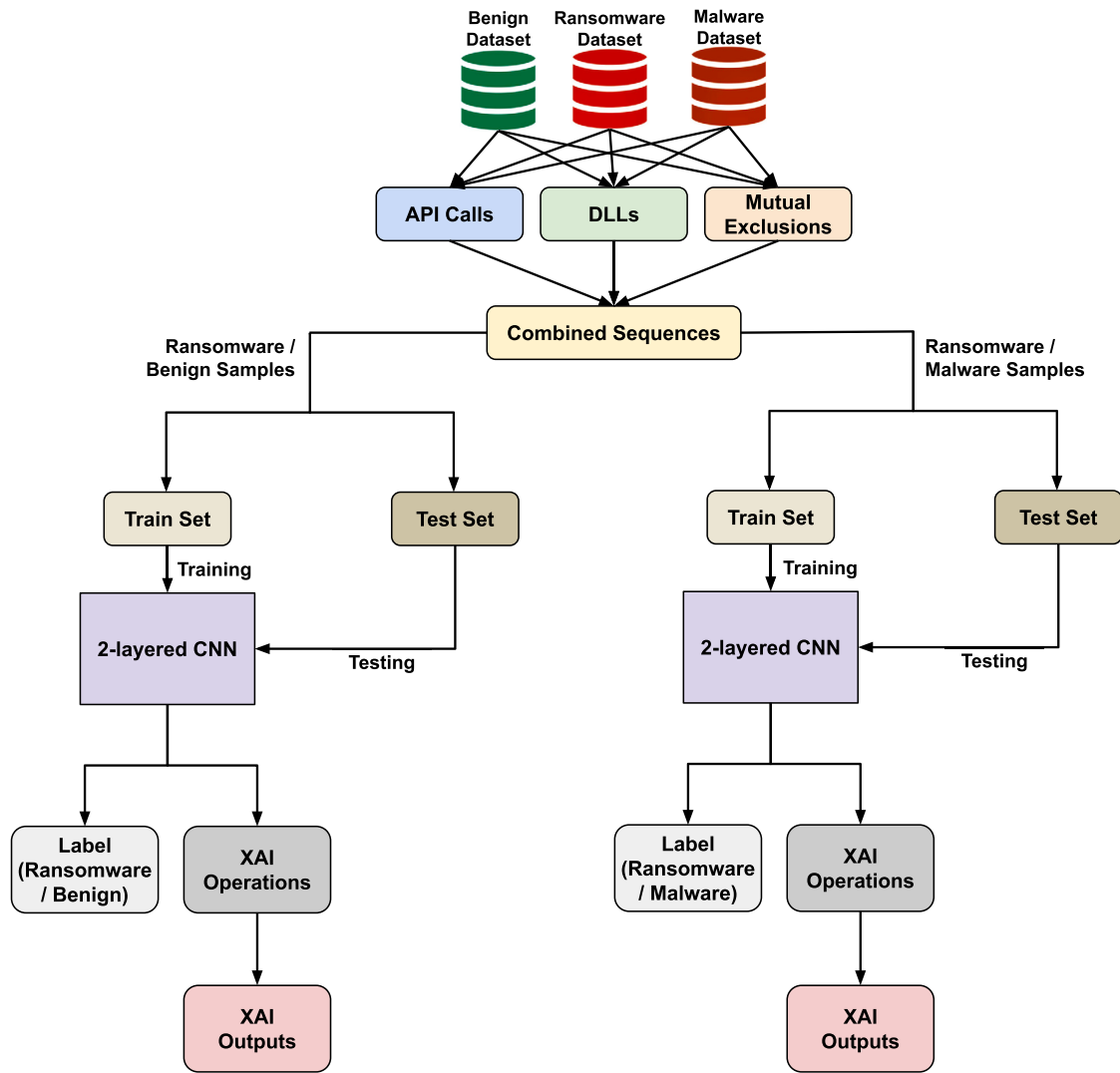


Fig. 1. General workflow of XRan.

ples. The detailed ransomware/benign classification framework is given in Fig. 2. The only differences in the classifications of ransomware/malware and ransomware/benign are the outputs of the model, and the datasets used in the training and testing processes. Each step visualized in these two figures is explained in detail in this section under different subheadings.

3.1. Feature extraction

In order to extract the characteristics of ransomware, each sample from each dataset was dynamically analyzed. For this purpose, a widely utilized dynamic analysis tool, Cuckoo Sandbox (2019), has been used. Cuckoo provides a report for the analyzed sample containing a large number of features that describe the actions taken or triggered by the software. In our previous work (Gulmez et al., 2023), we analyzed these features and demonstrated their effect on the performance of the detection model. In the light of the information we obtained in that study, we have selected three features as defined below.

- **API calls:** Since API calls can adeptly represent the behavior of a software, they are widely used in the detection and classification of ransomware and also malware in general. The malicious activities of a ransomware, e.g. communicating with the attacker, encrypting the files or locking the screen, can be identified through API calls.

- **DLLs:** Similarly to the API calls, DLLs are also quite useful in terms of deducing the possible activities of a software. An attacker can use DLLs for various reasons such as obfuscating the malicious activities in the source code or accessing specific functions to perform malignant actions.
- **Mutual Exclusions:** A ransomware can use mutual exclusions with the intent of preventing users from accessing their own files. Thus, such an activity could be distinctive in terms of the detection of ransomware.

After Cuckoo analysis, API calls, DLLs and Mutexes obtained from each executable file are evaluated as a set of features of that file. Each feature set is represented as a data sequence that is generated by chronologically ordering the items in that set. Then these sequences are concatenated in order to form a single sequence that describes the behavior of the software. However, in order to feed a deep learning algorithm, the sequence length has to be pre-defined. Thus, we have restricted the length of each feature in the combined sequence. The formal statement of this sequence is $A_a || D_d || M_m$ where A , D , and M are the sequences of API calls, DLLs, and mutual exclusions and the a , d , and m are the lengths of them respectively. The generation of the combined sequences has the complexity of $O(n * (a + d + m))$, where n is the number of samples, which converges to $O(n)$ since a , d , and m are pre-defined constants.

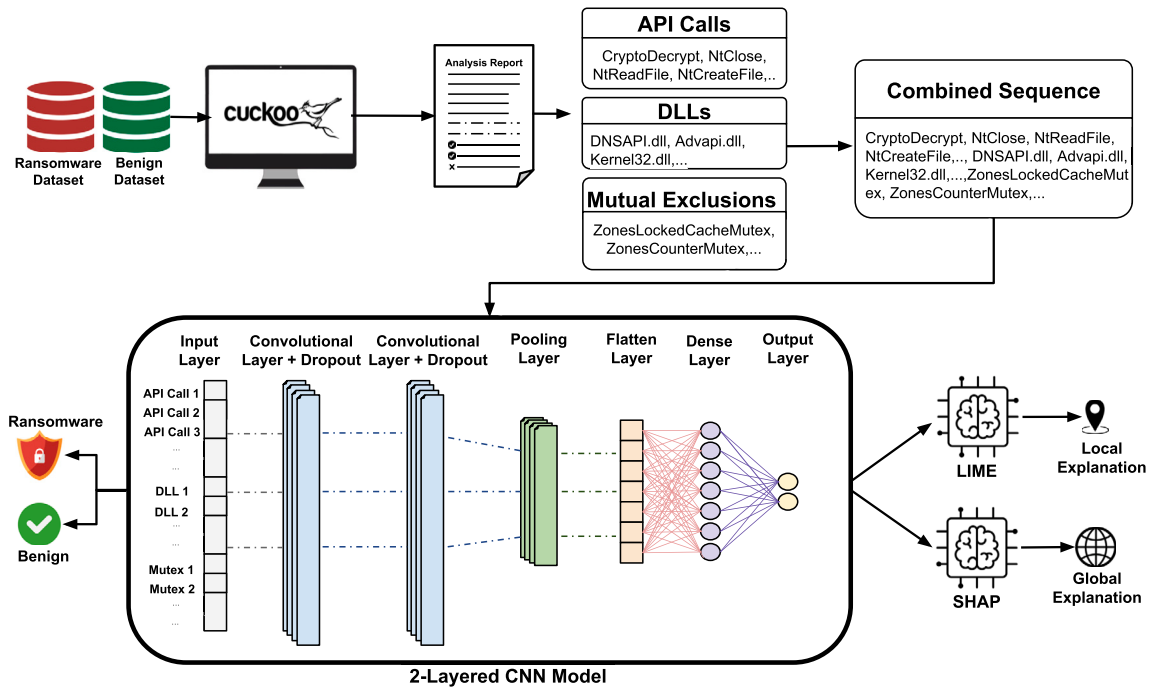


Fig. 2. Detailed ransomware/benign classification framework of XRan.

The dynamic analysis phase requires more time and resource consumption for each sample compared to static analysis. This is a common problem with all dynamic analysis-based malware detection techniques. However, despite these disadvantages, dynamic analysis is an important process that security professionals, incident response teams, product developers, and organizations use to provide a solid defense against cyber threats. Despite the resource and time consumption associated with dynamic analysis, the insights gained contribute to risk reduction. By understanding the behavior of malware, organizations can implement targeted security measures to reduce the impact of potential attacks. In particular, dynamic analysis becomes a necessity when working on software samples designed to bypass static analysis through techniques such as encryption, obfuscation, and packaging, as these methods significantly degrade the performance of detection techniques based on static analysis.

3.2. Deep learning

Both machine and deep learning algorithms are widely adopted to automate the detection and/or classification of ransomware. In this paper, we have proposed a detection model based on CNN, which is a well-known and widely used deep learning algorithm. Although it was originally designed for image processing tasks, CNN is also quite efficient with sequential data with its ability to extract underlying relationships just as if they are some objects in a picture to be detected. A CNN architecture consists of four main layers: convolutional layer, activation layer, pooling layer, and fully-connected layer.

Convolutional Layer: In this layer, a set of small-sized filters (kernels) is employed. The filtering operation involves sliding these kernels across the input data, performing element-wise multiplication at each position. This process generates a feature map that effectively involves local patterns within the input data.

Stride defines the number of steps to slide the kernel at each step. The input's response to a kernel forms a feature map that represents the value of the representative level of each feature on the input itself. In other words, each element in a feature map describes how much of each part of the input represents the whole input.

Activation Layer: After the convolution operation, an activation function performs element-wise operations with the purpose of capturing

complex relationships. In our architecture, *Sigmoid* has been selected as the activation function since it works well with binary classification. The formula of the sigmoid function is given in Equation (1). Sigmoid function has a range of [0, 1].

$$S(x) = \frac{1}{1 + e^x} \quad (1)$$

Pooling Layer: Pooling is a downsampling technique that is used for the reduction in the dimension of feature maps, leaving only the most important features and improving the computational efficiency of the model. Depending on the architecture and the data, there are several pooling operations. Our architecture employs *Max Pooling* as it helps with hierarchical feature learning. The formula of the Max Pooling is given in Equation (2).

$$MP(x)_{i,j} = \max_{p,q \in Pool(i,j)} x_{p,q} \quad (2)$$

where i and j are the indices of a pool, p and k are the indices of the maximum element within that pool.

Fully-Connected Layer: A fully connected layer refers to a neural network configuration in which each neuron performs a transformation on the input vector using a weight matrix. This layer (also known as *Dense layer*) connects the neurons of the previous layer to the neurons of the succeeding layer. In this way, we obtain a mapping between the input and the output layer.

3.3. Explainable artificial intelligence models

XAI, an abbreviation for Explainable Artificial Intelligence, encompasses a set of tools utilized to thoroughly interpret and comprehend predictive models generated through machine learning and deep learning algorithms. In the pursuit of achieving high accuracy in learning and prediction for various complex problems, deep learning architectures have grown increasingly intricate, making it challenging for both users and experts to analyze the decision-making stages of these models. In this context, XAI plays a vital role as part of broader human-centric responsible AI practices, shedding light on potential biases, expected impact, and explainability of the model. XAI effectively bridges the gap between deep learning models and human systems. In our research, we

Table 2
Dataset properties.

Dataset Name	Source	Year	Type	Extension	# of Samples	Average Analysis Time (seconds)
RD1	VirusShare (VirusShare, 2016)	2016	Ransomware	.exe	6,263	217.4
RD2	Sorel-20M (Harang and Rudd, 2020)	2020	Ransomware	.bin	7,703	135.7
RD3	ISOT (Jethva et al., 2020)	2020	Ransomware	.json	668	1504.4
MD	VX Heaven (VX Heavens, 2010)	2010	Malware	.exe	6,263	86.5
BD	Windows System Files, Download.Com (CNET, 2023), Software.Informer.Com (Software Informer, 2016), and PortableFreeware.Com (Portable Freeware, 2023)	2019-2020	Benign	.exe	14,797	107.5

have employed two specific XAI techniques, namely LIME (Ribeiro et al., 2016) and SHAP (Lundberg and Lee, 2017), to elucidate the feature importance and underlying abstractions within the model. LIME, which stands for Local Interpretable Model-Agnostic Explanations, provides explanations on a per-prediction basis using a local and interpretable model. Unlike offering a general explanation of the model's common decision mechanisms learned from all data, LIME explains how a specific observation is categorized based on its local features. In contrast, SHAP, which stands for SHapley Additive exPlanations, takes a different approach by calculating Shapley values. These values represent the average of the marginal contributions of all possible features, taking into account the contribution of each feature to the final decision. This allows SHAP to reveal the overall contribution of features to the model's decision-making process, going beyond merely providing local explanations.

In a formal context, the explanations generated by the LIME technique for a given observation x can be obtained through the following equation:

$$\phi(x) = \operatorname{argmin}_{g \in G} \delta(f, g, \varphi_x + \Omega(g)) \quad (3)$$

where $g \in G$ corresponds to an individual interpretable model from G which is a set of interpretable models (e.g. decision tree, convolutional neural network). $\Omega(g)$ represents the complexity of the explanation of all $g \in G$. φ_x refers to a proximity weight between the prediction of the explanation model g and the original model f . The goal is to minimize the loss function δ that measures the proximity between the explanation model and the original model for the specific observation.

For a given model f , the Shapley values are computed as follows:

$$\zeta_i(f, x) = \sum_{s \subseteq F} \frac{|s|!(M - |s| - 1)!}{M!} [f_x(s \cup i) - f_x(s)] \quad (4)$$

where $\zeta_i(f, x)$ is the Shapley value of feature i of a given observation x for model f , s represents the subset of features F that is the set of all features in the original set.

4. Experimental results

4.1. Dataset description and experimental setup

In this work, five datasets were used with the purpose of obtaining dataset-independent results. These datasets are explained below. Table 2 provides information about the source from which the data sets were collected, publication years, dataset types, the number of samples they cover, and average dynamic analysis times per sample.

- **RD1 (Ransomware Dataset 1):** The dataset contains 6,263 executable crypto-ransomware samples from VirusShare website (VirusShare, 2016).
- **RD2 (Ransomware Dataset 2):** The dataset contains 7,703 binary ransomware samples from Sorel-20M dataset (Harang and Rudd, 2020). Despite the dataset containing more ransomware samples, we have filtered them considering their tags. According to the

explanation in the research study this dataset was published on Harang and Rudd (2020), the tags correspond to the number of tokens identified as a specific malware class such as adware, worm, ransomware, etc. Since this paper aims at the detection of ransomware, we have selected samples with relatively more ransomware tags and fewer other malware tags. As a result, we have downloaded 7,703 ransomware samples.

- **RD3 (Ransomware Dataset 3):** The dataset contains 668 ransomware samples from ISOT ransomware dataset (Jethva et al., 2020). The samples are from 21 different Ransomware families. It must be noted that this dataset contains dynamic analysis JSON reports only, not the executable files.
- **MD (Malware Dataset):** The dataset contains 6,263 malware samples from various malware families such as virus, worm, trojan, etc. except ransomware from VX Heaven dataset (VX Heavens, 2010). This dataset was used to test XRank's ability to detect ransomware among other malware families.
- **BD (Benign Dataset):** The dataset contains 14,797 benign samples from various categories such as tools, games, drivers, system files, etc. The samples were collected from Download.Com (CNET, 2023), Software.Informer.Com (Software Informer, 2016), PortableFreeware.Com (Portable Freeware, 2023) websites, and Windows System Files.

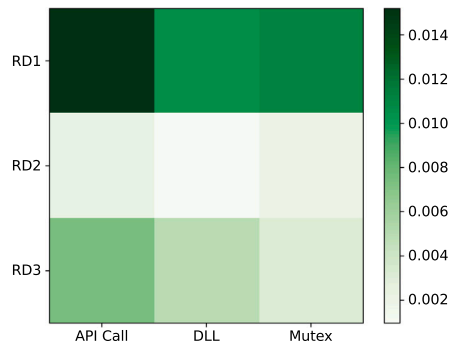
XRank was trained and tested on RD1, RD2, and RD3 separately to provide a fair comparison with the methods using one of these datasets. In addition, we created a dataset by combining these datasets, i.e. RD1 + RD2 + RD3 + BD for ransomware/benign classification and RD1 + RD2 + RD3 + MD for ransomware/malware classification, in order to improve its generalization ability and prove that XRank can work on a dataset consisting samples with diverse characteristics. Additionally, it must be noted that we have implemented randomized undersampling on BD and MD for the individual tests in order to work on a balanced dataset. For instance, to train a model on RD3 and BD, we used all of the samples from RD3 and randomly selected 668 benign samples from BD.

We analyzed these data sets to observe the relationship between them based on common unique features in the feature sets, with the idea that this might be useful in providing interpretations of the results found by the proposed model. To observe the similarity of samples within the ransomware datasets to the benign dataset, and similarly to the malware dataset, we used a metric called the *Coverage Percentage (CP)* as below:

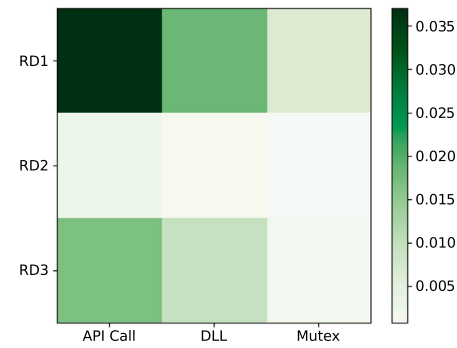
$$CP_f(D_1, D_2) = \frac{|U_f(D_1) \cap U_f(D_2)|}{|U_f(D_2)|} \quad (5)$$

where $CP_f(D_x, D_y)$ is the coverage percentage of Dataset x over Dataset y for the feature set f and $U_f(D_x)$ is the unique feature set of Dataset x for the feature set f .

By calculating CP, we have measured the percentage of a ransomware dataset to cover the unique features in a benign or malware dataset. The results of these calculations are presented visually as heatmaps in Fig. 3. According to Fig. 3a, RD1 contains 1.4% of the



(a) CP values of each ransomware dataset over BD for each feature set.



(b) CP values of each ransomware dataset over MD for each feature set.

Fig. 3. Heatmaps of CP values of each ransomware dataset over BD and MD. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

unique API Calls from BD while RD2 covers only 0.02% of the unique DLLs used by the samples of BD. Based on the information given in Fig. 3a, it is observed that RD1 is the most similar dataset to the BD among the three ransomware datasets we have used. This information indicates that RD1 might have more similar sequences to the samples of BD since they have a relatively large number of common features while RD2 seems to be mostly distinct and thus, has less similar sequences to the samples of BD. In Fig. 3b, the CP values of ransomware datasets over MD have been demonstrated. Similarly, RD1 seems to have the largest number of common features with MD considering all of the feature sets. RD2 has the most distinct feature sets from both BD and MD, followed by RD3. These results may provide insights into the anticipated difficulty level associated with classifying samples within the datasets. The observed commonalities and distinctions between ransomware datasets (RD1, RD2, RD3) and the benign (BD) and malware (MD) datasets offer a preliminary understanding of the classification challenges that may be encountered. Specifically, RD1 presents a nuanced scenario, sharing common features with both BD and MD, suggesting a potentially intricate classification task. On the other hand, RD2 and RD3 appear to exhibit more distinct characteristics, hinting at a comparatively less challenging classification process. These findings contribute to an initial assessment of the classification difficulty inherent in the datasets under consideration.

All of the experiments were executed on a computer with a Core i7 9700K CPU and 64 GB memory running 64bit Ubuntu 20.04.4 LTS. The Cuckoo Sandbox was installed as virtual machine with Windows 7 as the operating system. Since RD3 dataset contains Cuckoo analysis reports instead of the ransomware itself, we aimed at forming as similar as possible test environment for the other datasets we have used. Thus, we have defined the upper limit of the time period to analyze each sample as 60 minutes as it was defined by the authors of the study (Jethva et al., 2020) which is the source of RD3. Defining a high upper limit for the analysis process is a commonly adopted strategy by researchers (Shaukat and Ribeiro, 2018; Jethva et al., 2020; Wu et al., 2012; Shibahara et al., 2016, 2019), usually with an attempt to detect a malicious software with a timer. However, most of the samples terminate within 1-4 minutes and the analysis process comes to an end. Thus, the weighted average of the analysis time durations of all datasets in this study was shaped as 155.3 seconds. The individual analysis time information of each dataset is also given in Table 2.

The proposed model employs a 2-layered CNN architecture with Dropout and MaxPooling layers. Binary CrossEntropy loss function and Sigmoid activation function were used with the optimizer Adam. To demonstrate the efficiency of this architecture, we have tested 1-layered CNN as well as several machine learning algorithms and LSTM on the same datasets. Machine learning algorithms were trained and tested through scikit-learn library provided for Python, with the default pa-

rameters. LSTM has a 2-layered structure as CNN, using the same functions and optimizer. The sequence length was defined as 520, containing 500 API calls, 10 DLLs, and 10 mutual exclusions. The detailed analysis of these parameters is provided in Section 4.3. In order to train and test each model, the dataset was divided into three sub-datasets: training data, validation data, and testing data with a distribution of 80%, 10%, and 10% respectively. 10-fold cross-validation was used to obtain accuracy results of the model for different data subsets, and the final accuracy values correspond to the average result of values obtained from all folds.

4.2. Detection performance evaluation

In this section, the performance of the proposed method XRun is evaluated by comparing it with baseline methods, and the state-of-the-art methods on ransomware detection. We also evaluate XRun with respect to ransomware/malware classification. To provide a fair comparison opportunity to other researchers, the test results over individual datasets are also provided, despite the fact that XRun's detection model is trained and tested on the combination of 3 ransomware datasets.

4.2.1. Ransomware-benign classification

The proposed method XRun's basic approach is to combine different attribute spaces that represent different perspectives of an executable. The process of combining these feature spaces is at the sequence level. In order to reveal the contribution of this merging process to the ransomware detection process, a 2-layered CNN included by XRun was performed on each different attribute set (Gulmez et al., 2023) and the accuracy values obtained are shown in Table 3.

API calls are one of the predominant dynamic attributes utilized for the identification of malware. Therefore, researchers commonly employ this feature set for the ransomware detection as well. However, Table 3 indicates that the model's overall accuracy tends to stabilize within the range of 85 – 88% when using only API calls. These observed low accuracy results for all datasets are due to the high False Positive Rate (FPR), which indicates the rate of false alerts. Conversely, when focusing specifically on RD1 and RD3 datasets, it is observed that the mutual exclusions-based detection yields lower TPRs, and also lower FPRs. In DLL-based detection, more balanced TPR and FPR results are obtained compared to these two feature sets. When detection is performed using our feature set that consists of the concatenated sequences, a balance is achieved for all three datasets at a more expected level compared to the other feature sets, i.e. very high TPR and very low FPR results.

Detection results using different methods are presented in Table 4 for four datasets. The results show that XRun outperforms almost all baseline methods in terms of accuracy, TPR, FPR, and F-score. XRun provides 99.1% detection performance with 3.6% FPR for RD1 dataset,

Table 3

The performance of the 2-layered CNN model on different data sequences for ransomware/benign classification.

Datasets	API call sequence			DLL sequence			Mutex sequence			Merged sequence		
	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR
RD1 + BD	0.85	1	0.31	0.92	0.95	0.12	0.94	0.88	0.01	0.98	0.99	0.04
RD2 + BD	0.88	1	0.24	0.99	0.99	0	0.99	0.99	0	1	1	0
RD3 + BD	0.85	1	0.25	0.94	0.96	0.11	0.93	0.88	0.01	1	1	0

Table 4

The performance of different methods on ransomware/benign classification.

Datasets	Detection Technique	Accuracy	TPR	FPR	F-Score
RD1 + BD	Decision Tree	0.887	0.909	0.123	0.893
	Random Forest	0.928	0.944	0.081	0.931
	Naive Bayes	0.611	0.556	0.358	0.596
	KNN (k = 3)	0.807	0.892	0.230	0.826
	KNN (k = 5)	0.786	0.890	0.255	0.811
	LSTM	0.952	0.981	0.070	0.955
	CNN	0.955	0.977	0.061	0.958
	XRan (2L-CNN)	0.976	0.991	0.036	0.977
RD2 + BD	Decision Tree	0.999	1.000	0.002	0.999
	Random Forest	1.000	1.000	0.000	1.000
	Naive Bayes	0.813	0.791	0.172	0.809
	KNN (k = 3)	1.000	1.000	0.000	1.000
	KNN (k = 5)	0.999	1.000	0.001	0.999
	LSTM	1.000	1.000	0.000	1.000
	CNN	1.000	1.000	0.000	1.000
	XRan (2L-CNN)	1.000	1.000	0.000	1.000
RD3 + BD	Decision Tree	0.970	0.971	0.029	0.971
	Random Forest	0.978	0.971	0.014	0.978
	Naive Bayes	0.597	0.643	0.392	0.625
	KNN (k = 3)	0.847	0.886	0.173	0.855
	KNN (k = 5)	0.843	0.886	0.173	0.855
	LSTM	0.970	0.971	0.029	0.971
	CNN	0.970	0.971	0.029	0.971
	XRan (2L-CNN)	1.000	1.000	0.000	1.000
RD1 + RD2 + RD3 + BD	Decision Tree	0.912	0.943	0.115	0.913
	Random Forest	0.964	0.963	0.036	0.964
	Naive Bayes	0.680	0.700	0.325	0.687
	KNN (k = 3)	0.869	0.906	0.156	0.874
	KNN (k = 5)	0.865	0.903	0.159	0.871
	LSTM	0.976	0.989	0.035	0.977
	CNN	0.976	0.988	0.035	0.976
	XRan (2L-CNN)	0.990	0.994	0.010	0.992

100% detection performance with 0% FPR for RD2 dataset, 100% detection performance with 0% FPR for RD3 dataset. For the fourth dataset, which is a combination of RD1, RD2, and RD3, XRan also achieves significantly high performance with 99.0% accuracy, 99.4% TPR, and 99.2% F-score. The detection results of the methods are summarized as follows:

- The results show that XRan outperforms all baseline methods for RD1 dataset. The performance of XRan is followed by that of CNN. The reason why the proposed method achieves higher performance than CNN is that XRan has a two-layered convolutional process. The proposed method might be more effective in extracting lower-level features since stacking convolutional layers provide a hierarchical decomposition of the sequence data. Similarly, for the fourth dataset, which contains all samples in the three datasets, XRan produces better accuracy, TPR, FPR, and F-score values, compared to CNN.
- XRan achieves 100% detection performance for RD2 and RD3 datasets. The same performance values are obtained using RF, KNN (k = 3), CNN, and LSTM for RD2 dataset. The results show that these two datasets have a better distinguishable feature space than

RD1 datasets, in other words, more distinctive sequence patterns. Especially in RD2 dataset, even RF algorithm that uses only frequency information can provide 100% detection performance. This inference about RD2 and RD3 also supports the similarity results of these two datasets to the BD as mentioned in Section 4.1.

- It is observed that deep learning architectures such as CNN and LSTM give better results than traditional machine learning methods such as DT, RF, KNN, and Naive Bayes, when examining the results based on the RD1 and RD2 datasets. The most important reason for this may be that classical machine learning methods ignore sequence information and evaluate the features separately. CNN and LSTM-based architectures perform a latent feature mapping by revealing correlations based on local and global order relations between attributes in API call, DLL, and mutex arrays.

4.2.2. Ransomware-malware classification

Within this section, we evaluate the effectiveness of XRan in the context of distinguishing ransomware from more generalized forms of malware. Fundamentally, when it comes to a new malware, XRan undertakes the task of determining whether it pertains to the category of ransomware or falls within the general malware. Table 5 depicts the

Table 5

The performance of different methods on ransomware/malware classification.

Datasets	Detection Technique	Accuracy	TPR	FPR	F-Score
RD1 + MD	Decision Tree	0.900	0.878	0.076	0.901
	Random Forest	0.921	0.892	0.048	0.921
	Naive Bayes	0.600	0.601	0.400	0.608
	KNN (k=3)	0.829	0.858	0.201	0.838
	KNN (k=5)	0.816	0.854	0.226	0.827
	LSTM	0.930	0.920	0.059	0.931
	CNN	0.931	0.961	0.100	0.935
	XRan (2L-CNN)	0.927	0.955	0.105	0.930
RD2 + MD	Decision Tree	1.000	1.000	0.000	1.000
	Random Forest	1.000	1.000	0.000	1.000
	Naive Bayes	0.952	0.972	0.074	0.959
	KNN (k=3)	1.000	1.000	0.000	1.000
	KNN (k=5)	1.000	1.000	0.000	1.000
	LSTM	1.000	1.000	0.000	1.000
	CNN	1.000	1.000	0.000	1.000
	XRan (2L-CNN)	1.000	1.000	0.000	1.000
RD3 + MD	Decision Tree	0.963	0.943	0.016	0.964
	Random Forest	0.978	0.971	0.016	0.978
	Naive Bayes	0.567	0.643	0.516	0.608
	KNN (k=3)	0.873	0.914	0.172	0.883
	KNN (k=5)	0.866	0.914	0.188	0.877
	LSTM	0.993	0.986	0.000	0.993
	CNN	1.000	1.000	0.000	1.000
	XRan (2L-CNN)	1.000	1.000	0.000	1.000
RD1 + RD2 + RD3 + MD	Decision Tree	0.939	0.950	0.085	0.957
	Random Forest	0.945	0.943	0.051	0.960
	Naive Bayes	0.650	0.697	0.461	0.736
	KNN (k=3)	0.888	0.934	0.222	0.921
	KNN (k=5)	0.876	0.934	0.259	0.914
	LSTM	0.956	0.947	0.023	0.968
	CNN	0.955	0.945	0.023	0.967
	XRan (2L-CNN)	0.956	0.947	0.023	0.968

Table 6

The performance outputs of the state-of-the-art methods for ransomware detection.

Paper	Analysis Technique	Dataset Source		TPR	FPR	ACC
		Benign	Ransomware			
Sgandurra et al. (2016)	Dynamic Analysis	Software.Informer.Com	VirusShare	0.9634	0.0161	N/A
Hasan and Rahman (2017)	Hybrid Analysis	N/A	VirusShare	0.96	N/A	0.961
Chen et al. (2017)	Dynamic Analysis	Software.Informer.Com	VirusShare	0.976	0.012	N/A
Shaukat and Ribeiro (2018)	Hybrid Analysis	N/A	VirusShare	0.9825	0.056	N/A
Hwang et al. (2020)	Dynamic Analysis	Softonic.Com	VirusShare	N/A	0.0483	0.9728
Qin et al. (2020)	Dynamic Analysis	Sangfor Technologies	Sangfor Technologies	N/A	N/A	0.959
Jethva et al. (2020)	Hybrid Analysis	Software.Informer.Com	ISOT	1.000	0.133	0.987
XRan	Dynamic Analysis	Windows System Files,	VirusShare	0.991	0.036	0.976
		Download.Com,	Sorel-20M	1.000	0.000	1.000
		Software.Informer.Com, and	ISOT	1.000	0.000	1.000
		PortableFreeware.Com	VirusShare, Sorel-20M, and ISOT	0.994	0.047	0.982

performance scores of different methods for ransomware/malware classification. The results demonstrate that XRan achieves higher accuracy results than baseline methods or comparable results. For RD1 dataset, the highest TPR value was obtained with CNN, and the lowest FPR value was obtained with RF. The performance of the proposed method is comparable to that of CNN, which achieved 95.5% TPR, 10.0% FPR. For RD2 and RD3 dataset, XRan achieves 100% TPR, 0% FPR. For the fourth dataset, which contains all samples in the three datasets, XRan obtains higher Accuracy, and TPR, and also lower FPR values than all baseline methods except LSTM which has the same performance as XRan.

As a result, examining the performance outputs presented in Table 4 and 5 reveals a discernible pattern: the boundary between ransomware and benign assets is more distinct than the difference between ransomware and other malware categories. This is because of the similarity between the malicious behavior exhibited by common malware and ransomware.

4.2.3. Comparison with the state-of-the-art methods

In this section, the proposed method is compared with some state-of-the-art methods that provide ransomware detection. Table 6 depicts the performance results of these methods as reported in their respective papers. For a fair comparison, we compared XRan with the techniques that utilize dynamic or hybrid analysis. The analysis technique and the dataset source of these studies are also given in Table 6. More details about these studies, such as the number of samples and the feature sets they used, were given in Section 2 and Table 1.

Table 6 indicates that XRan achieves better detection results than state-of-the-art methods. According to the TPR values, the closest result to XRan was obtained by the authors (Jethva et al., 2020) who performed three different machine learning algorithms and obtained a TPR of 100% with 13.3% FPR. However, XRan obtained 100% of TPR and 0 false alerts on the exact same dataset. It should also be noted that XRan achieves this performance just by using dynamic analysis,

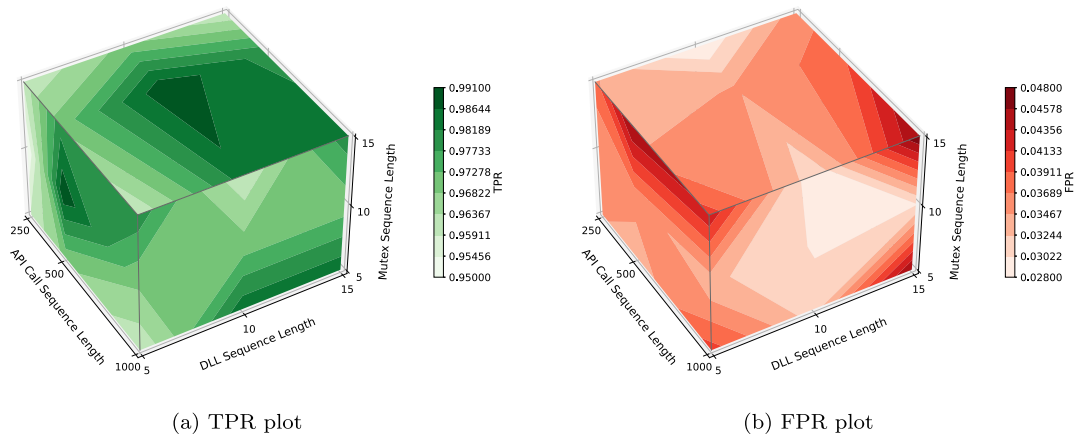


Fig. 4. 3-D plots of the parameter analysis on datasets RD1 and BD.

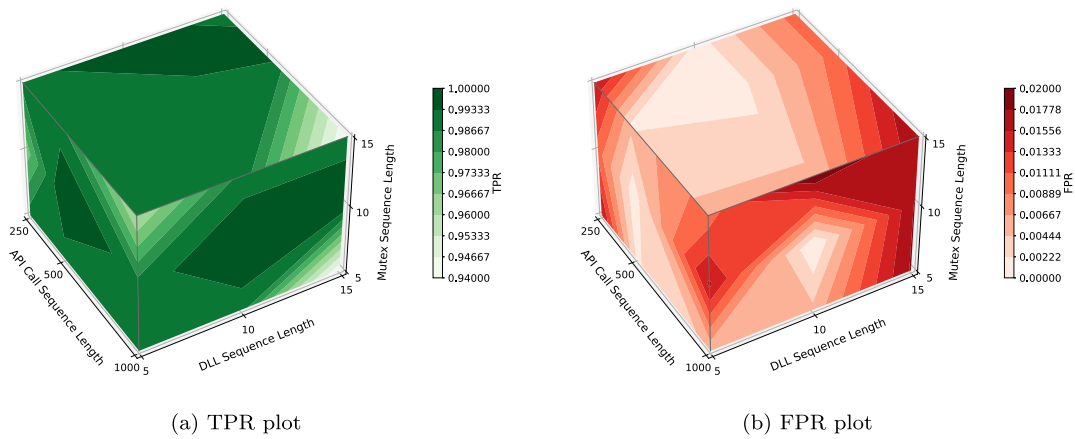


Fig. 5. 3-D plots of the parameter analysis on datasets RD3 and BD.

while the compared study implements hybrid analysis on a multi-stage architecture. The performance of this method is followed by another hybrid analysis-based method that provides a detection rate of 98.25% (Shaukat and Ribeiro, 2018). Their method focuses on using PE header, embedded resources, and strings. In terms of the attribute set they use, this method differs from XRan. In addition, the number of samples they use is 20 times less than the number of samples we use, which makes it questionable whether their methods can be generalized well to new ransomware samples. Similarly, the number of samples used by the methods presented in the articles (Sgandurra et al., 2016), (Hasan and Rahman, 2017), (Chen et al., 2017) is quite low. There is limited information about the overall performance of the methods, as the number and variety of samples are not sufficient. As an intersection with XRan, these three studies included API Calls in their feature sets. Also, they used a ransomware dataset obtained from VirusShare. However, the detection rate of XRan on VirusShare dataset outperforms them. The accuracy score given by Hasan and Rahman (2017) is 96.1%. XRan surpasses this performance with an accuracy score of 97.6% on the same dataset. The accuracy values of the methods proposed in the studies (Sgandurra et al., 2016) and (Chen et al., 2017) were not presented but the reported FPR values of these methods are lower than the FPR value of XRan. Test results show that XRan has more false alarms than these studies. However, as mentioned above, these studies were tested on small datasets. Hence, there is no proof that they will be able to keep FPR that low, with a larger dataset. Another study using VirusShare dataset is the study of Hwang et al. (2020). Despite using the same dataset, implementing the same analysis technique, and having common feature sets with XRan, this study also falls behind XRan, both in terms of accuracy and FPR. The study of Qin et al. (2020) is another dy-

namic analysis-based ransomware detection technique that focuses on API Call sequences. Although it benefits from CNN-based deep learning model for the classification task, just as in XRan, this study is also outperformed by XRan. The most important reason for this may be that it uses only the API call sequences in the learning phase. XRan, on the other hand, includes both API call sequences, DLL and Mutex sequences in the learning process.

4.3. Parameter analysis

As explained in Section 3.1, we have concatenated the sequences of three feature sets and obtained the combined sequence, $A_d||D_d||M_m$. In order to define the length of the sequences, a , d , and m , we have analyzed each dataset and calculated the geometric mean of each feature set. With respect to this analysis, we have found out that the average lengths of the sequences of API calls, DLLs, and mutual exclusions are 10,471.9, 11.1, and 10.7 respectively. Hence, we have tested the DLL and mutual exclusions with the length in a range of [5,15]. For the API calls, considering the learning capacity of the models, we have conducted our tests up to the length of 1,000. We set the lower bound as 250 since the shorter sequences didn't have a significant effect on the models.

The parameter analysis tests were realized for each ransomware dataset, in order to find and diagnose the similarities between them and specify the most optimum parameter set. Fig. 4 and 5 visualizes the results of these tests conducted on RD1, and RD3 respectively. RD2, on the other hand, was classified with 100% accuracy for the parameters within the given ranges, thus, we didn't provide an analysis for this dataset.

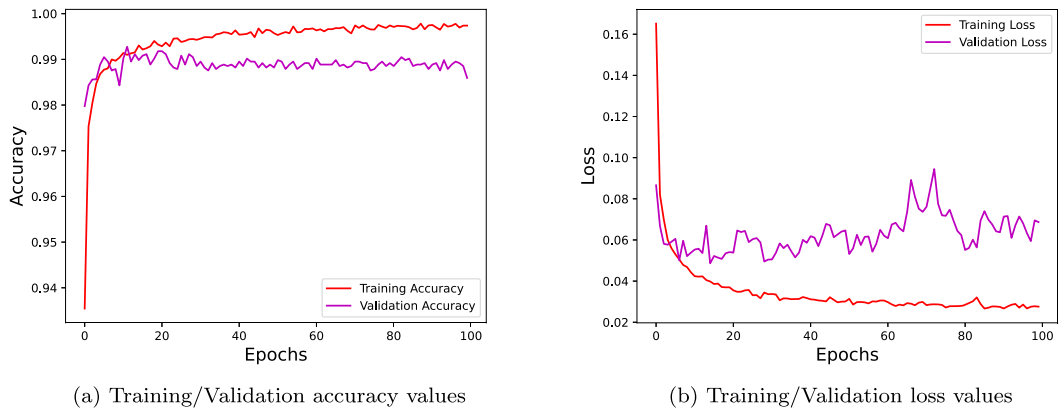


Fig. 6. Accuracy and loss performances of the detection model trained on RD1+RD2+RD3+BD.

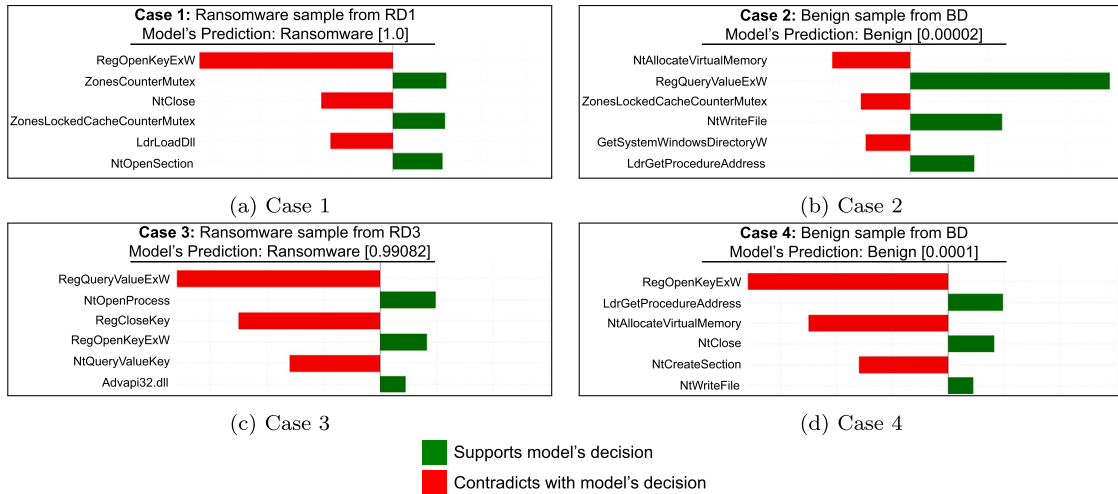


Fig. 7. Ransomware/Benign classifier outcomes: explanation of the correctly classified samples.

As seen in Fig. 4a and 5a, RD1 and RD3 reach the highest TPR with an API call sequence length of 500 and DLL sequence length of 10. These are the only common parameters for these two datasets to detect as many ransomware as possible. In terms of FPR, Fig. 5b demonstrates RD3 is prone to raise more false alarms when API call sequence length is higher than 500 or the DLL sequence length is 15. On the other hand, RD1 seems to have a higher FPR when the Mutex sequence length is 5 or 15 as seen in Fig. 4b.

To mitigate the risk of overfitting in the detection model, we conducted supplementary tests. As depicted in the accuracy and loss graphs presented in Fig. 6, the loss of the model increases noticeably after approximately 40 epochs. Therefore, we have defined the epoch parameter of XRan as 10 to make sure the model is not overfit.

4.4. Explanation of the detection model

4.4.1. Local explanation via LIME

As specified in Section 3.3, LIME provides per-prediction explanations instead of a global interpretation. Thus, we employed LIME to explain the decision-making process of the models on a few randomly selected samples. These explanations are visualized in Fig. 7, 8, 9, and 10. Since the models' output for a sample is a float value in a range of [0, 1], where 0 and 1 denote benign/malware and ransomware respectively, we also provided this value in the figures for a better understanding of the confidence level of the model's decisions.

Fig. 7 illustrates the test samples that had been correctly classified by the ransomware/benign classifier models. In Fig. 7a, we see that mutual exclusions, *ZonesCounterMutex* and *ZonesLockedCacheCounterMutex*,

become prominent with regard to labeling the sample as ransomware. Mutexes are used to synchronize access to shared resources in multi-threaded or multi-process environments to prevent issues such as data corruption, race conditions, and unexpected behavior, thus ensuring consistency and accuracy. Ransomware perpetrators leverage mutexes for diverse objectives. The *ZonesCounterMutex* is employed to regulate access to data within distinct program segments, while the *ZonesLockedCacheCounterMutex* serves to constrain access to a cache-associated counter across various regions. These two mutexes can ensure that only one instance of the malware is executed at a given time, which can help prevent detection by security utilities that monitor anomalous behavior. Using mutexes provides the controlled synchronization of cache updates, thus different threads or malware instances do not overwrite each other's data. These mutual exclusions are generally used to lock a specific location and were detected as being used in malicious software previously (Jackson, 2007; SonicWALL Security Center, 2020). *ZonesLockedCacheCounterMutex* is also observed in Fig. 7b as it contradicts the model's decision, which is benign. *ZonesLockedCacheCounterMutex* may also be observed in a benign sample. This mutex must have been mostly observed in ransomware samples, which contradicts the model's decision. However, the observation of this mutex does not cause the model to incorrectly label this sample. The model decides according to the frequency of use of an attribute that can be described as suspicious in that sample or the distribution of other attributes that are observed together.

NtAllocateVirtualMemory also seems to be a ransomware indicator as it contradicts with the model's decision of labeling the sample as benign in Fig. 7b and 7d. This information points out the ransomware's

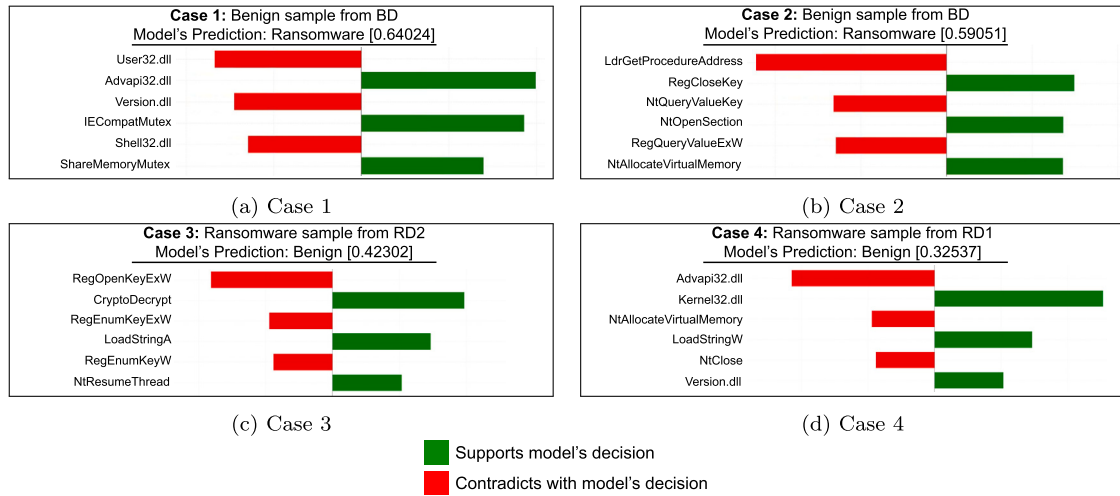


Fig. 8. Ransomware/Benign classifier outcomes: explanation of the misclassified samples.

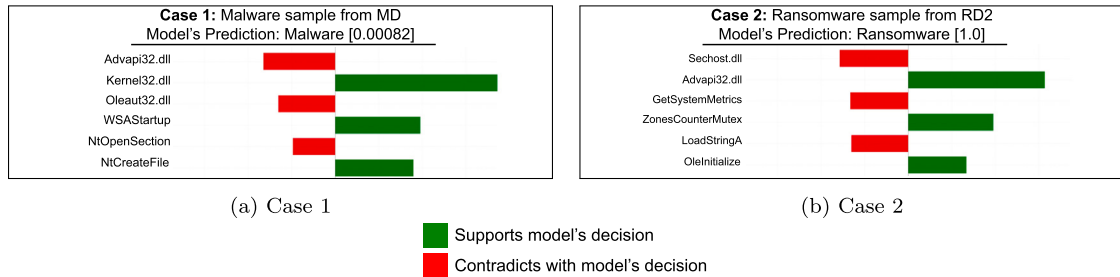


Fig. 9. Ransomware/Malware classifier outcomes: explanation of the correctly classified samples.

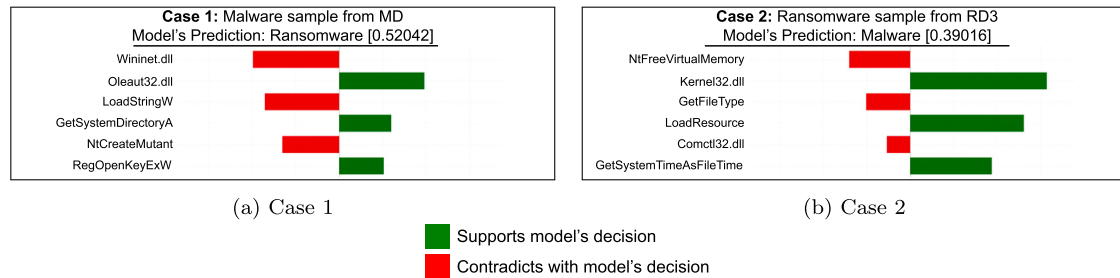


Fig. 10. Ransomware/Malware classifier outcomes: explanation of the misclassified samples.

inclination of manipulating memory for malicious activities such as code injection, access denial, system modifications, etc. *NtAllocateVirtualMemory* is a function specifically related to memory management within the Windows kernel provided by the Native API (NtAPI) of the operating system. Ransomware needs memory usage to perform encryption operations such as generating encryption keys. By using *NtAllocateVirtualMemory*, ransomware creates new memory regions during encryption operations to reduce the possibility of detection due to potentially excessive memory usage. In addition, ransomware might often use this function to scan the memory to identify valuable data required for encryption purposes and store data obtained during these scans.

Advapi.dll, that can be seen in Fig. 7c, is a dynamic link library (DLL) file in the operating system that contains various code and data related to advanced security-related services and processes such as user authentication, access control, and registry manipulation, is another ransomware representative. It is a DLL that is particularly likely to be observed in ransomware because it provides functions related to cryptographic operations, including hashing, encryption, and decryption. This information also matches up with the effect of the usage of API call *Reg-*

OpenKeyExW that is provided by *Advapi.dll*. *RegOpenKeyExW* provides the ability to open or modify registry keys, bypass the security countermeasures, or make the software persistent in the system which makes it quite beneficial for a ransomware. As expected, *RegOpenKeyExW* claims that the sample is a ransomware in Fig. 7c, and 7d. However, in Fig. 7a, *RegOpenKeyExW* acts conversely and contradicts with the model's decision to label the sample as ransomware. Such behavior highlights the importance of analyzing the correlation between multiple features instead of considering them individually.

NtOpenProcess function supports the model's decision as ransomware for Case 3, while *NtAllocateVirtualMemory* contradicts the model's decision as benign for Case 2 and 4. *NtOpenProcess* and *NtAllocateVirtualMemory* functions are part of the Windows Native API, which is a lower-level interface used by Windows to manage various system functions. Ransomware has been known to leverage these functions for unauthorized activities. Ransomware might use these functions for file manipulation (encryption) and managing memory regions. These functions might grant ransomware the ability to evade certain security tools that concentrate on monitoring higher-level activities.

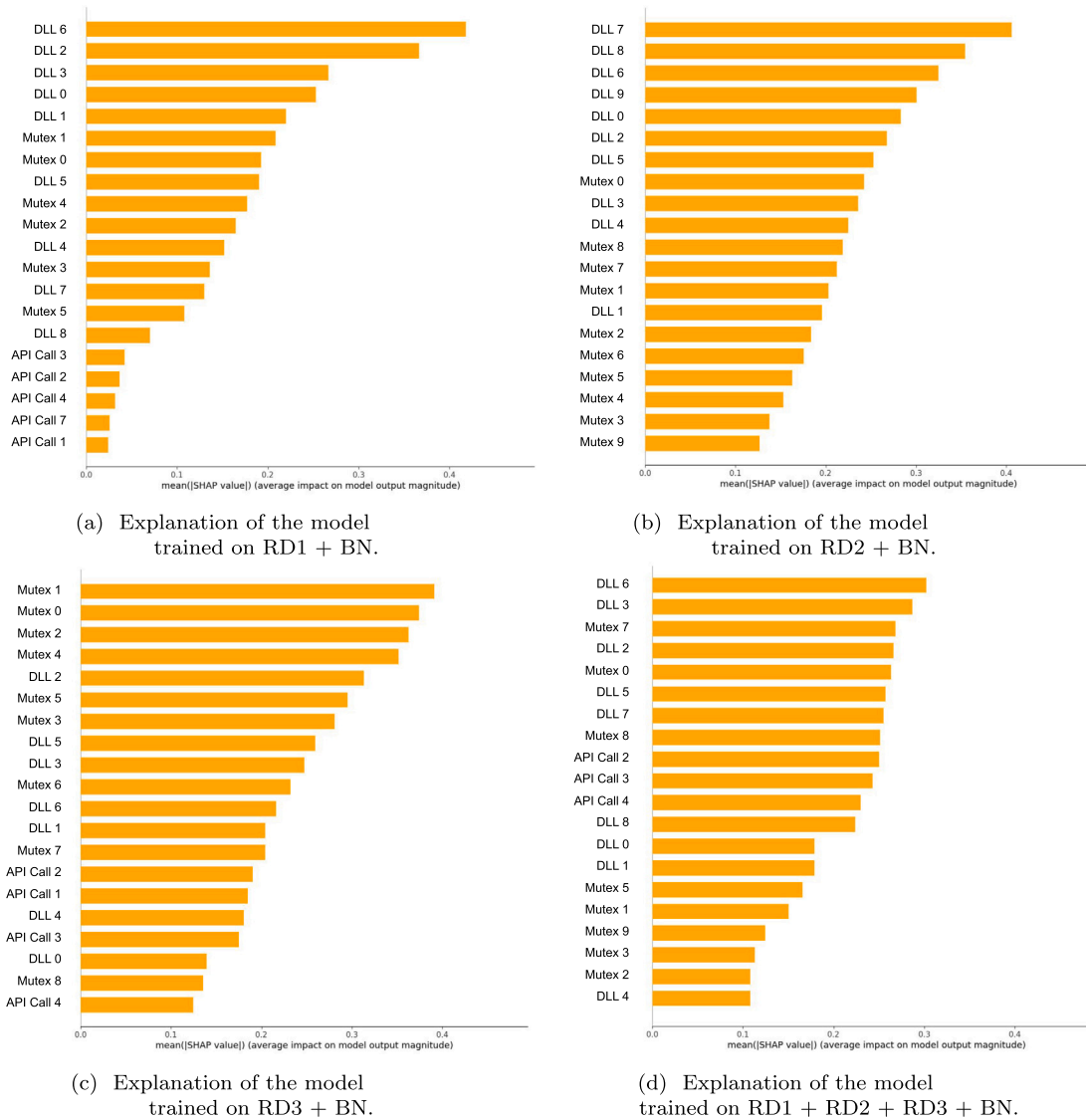


Fig. 11. Location-based global explanation of the ransomware/benign classifier models.

Fig. 8 represents the visual explanation of the misclassified samples from the ransomware/benign classifier models. As seen, *Advapi.dll* cause the misclassification of a benign sample in Fig. 8a as ransomware. It also contradicts with the model's decision as it labels a ransomware sample as benign in Fig. 8d. This situation supports the deduction of *Advapi.dll* is more likely to be used in ransomware samples. Similarly, *NtAllocateVirtualMemory* is one of the main reasons to classify a benign sample as ransomware in Fig. 8b. On the other hand, in 8d, *NtAllocateVirtualMemory* contradicts the model's decision of misclassifying a ransomware. In Fig. 8c, we observed a different API call, *CryptoDecrypt*, as the main reason for labeling a ransomware as benign. Naturally, ransomware is expected to crypt files, not to decrypt. Such behavior misguides the detection model into missing out a ransomware. *LoadStringA* and *LoadStringW* also have a similar effect as seen in Fig. 8c and 8d. These API calls are two variants of *LoadString* that are commonly used in benign applications with the purpose of loading a string from a library or an executable file. It is not unexpected that techniques involving API calls and dynamic-link library (DLL) usage, commonly seen in legitimate software, are also prevalent in ransomware. Ransomware creators might intentionally mimic typical behaviors exhibited by non-malicious programs to evade detection by security measures.

Since XRank is able to detect ransomware among other malware families, we have also provided an explanation of the ransomware/mal-

ware classification models. Fig. 9 and 10 represent the visualization of correctly classified and misclassified samples respectively by the classifiers which were trained on ransomware and malware datasets. The most striking information that can be derived from Fig. 9 and 10 is that the features which had been related to benign samples in ransomware/benign classification models, are mostly associated with malware samples in ransomware/malware classification task. For instance, *LoadStringA*, *Kernel32.dll*, *Version.dll*, and *LoadStringW* were some of the benign indicators in Fig. 7 and 8. To put it in a different way, these explanations imply that a malware looks like a benign software next to ransomware. In addition to this, some features which could be used in a non-ransomware malware are mostly linked with ransomware class, e.g. *Advapi32.dll*, *Oleaut32.dll*, and *Comctl32.dll*.

Oleaut32.dll provides Objective Linking and Embedding (OLE) services such as data encryption, file operations, and communication between applications and users. These functionalities are useful for any malware family but encryption and file operation seem to be distinctive functionalities as they ease the activities of a ransomware. Similarly, *Comctl32.dll* is known for its role in Graphical User Interface (GUI) management and can be used by any malware sample especially to mislead the users. But, manipulating GUI becomes more substantial for a ransomware with the purpose of creating dialog boxes to demand a ransom or locking the screen to block user's access to their files.

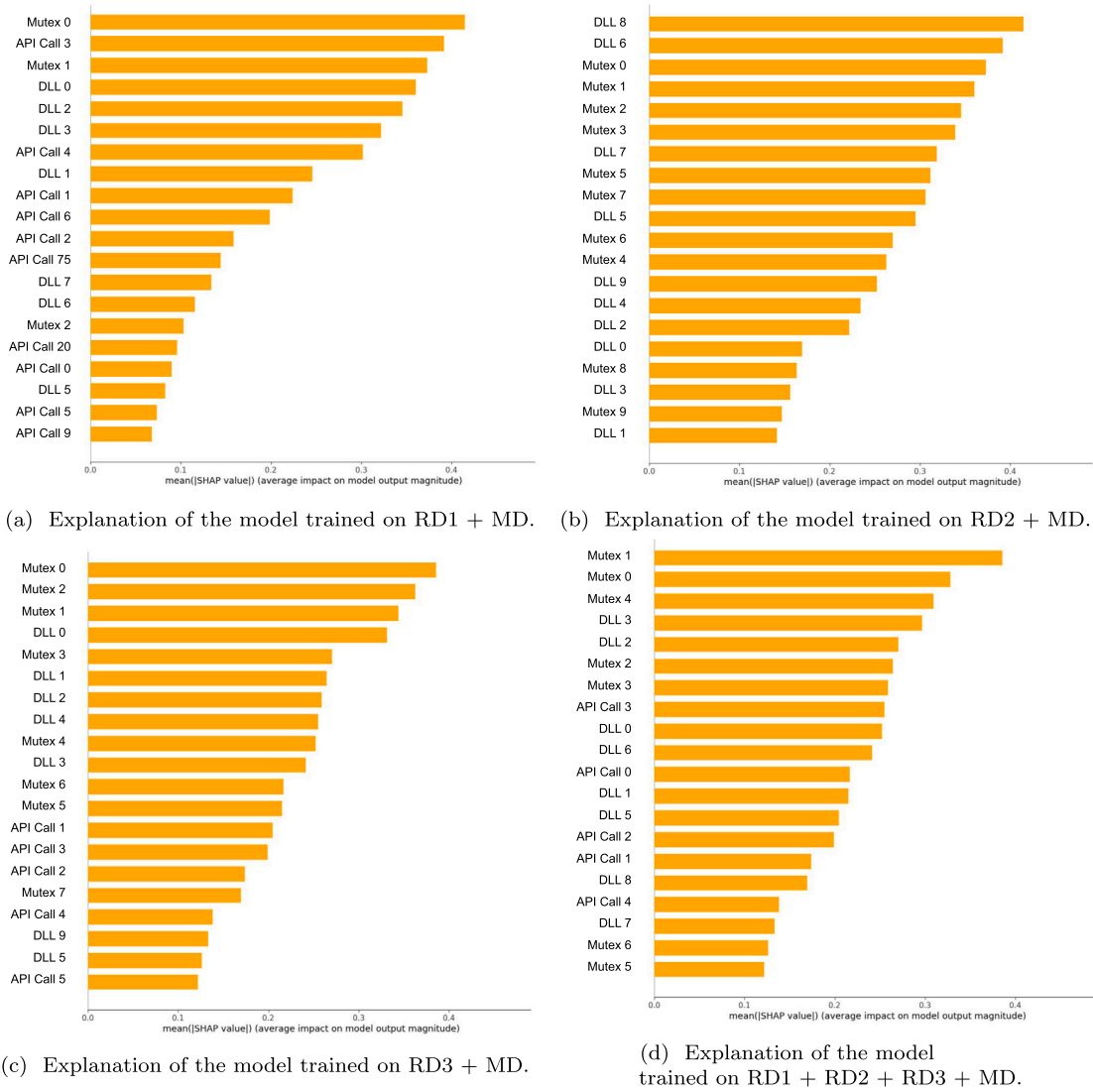


Fig. 12. Location-based global explanation of the ransomware/malware classifier models.

In Fig. 9b, *Sechost.dll* is seen as it contradicts with model's decision of labeling the sample as ransomware. *Sechost.dll* is a dynamic link library that provides security host services. By using this library, a malware can manipulate the authentication processes, escalate its privilege, propagate itself, extract sensitive information, and a lot more malicious activities. Naturally, such operations could be used by a ransomware, a malware family, but considering the main purpose of a ransomware and other malwares, *Sechost.dll* becomes more related to a non-ransomware malware.

Fig. 9a also reveals two more malware-related features: *Wininet.dll* and *NtCreateMutant*. *Wininet.dll* is a library with various network functions such as connecting to the internet or transferring data over HTTP. A ransomware could possibly use such a library with the purpose of approaching the Command and Control (C&C) servers in order to communicate with the attacker. But, *Wininet.dll* also provides the opportunity to spread and propagate the software, download additional malicious files, steal the login credentials, etc. *NtCreateMutant* can also be used for propagation, which makes it more related to non-ransomware malware samples. These two features, along with the usage of *LoadStringW*, contradict with the model's incorrect decision on labeling the malware sample as ransomware in Fig. 9a. However, *Oleaut32.dll*, *GetSystemDirectoryA*, *RegOpenKeyExW* and other ransomware-coded features misguide the model and misclassification occurs. Similarly, *NtFreeVirtualMemory*, which is a commonly used feature by ransomware samples

with memory manipulation purposes, contradicts with the decision of the model to make an incorrect decision in Fig. 10b but other features, especially *Kernel32.dll*, cause another misclassification.

4.4.2. Global explanation via SHAP

In addition to the local explanations, we have also provided the global interpretation of the models by employing SHAP. These explanations involve both location-based and feature-based interpretations. Location-based interpretations, visualized in Fig. 11 and 12, express the significance of a feature's location within the combined sequence. Such a statement provides information about the importance of the chronologic order of features, as well as the predominant feature sets among the three of them. Since XRan was trained on various datasets, these explanations were also provided for each of them. Fig. 11 and 12 represent the explanation of ransomware/benign classifiers and ransomware/malware classifiers respectively. For a better understanding, it must be noted that the features have 0-based indexing and so, a combined sequence is encoded as $[API_Call_0, API_Call_1, \dots, API_Call_{499}, DLL_0, DLL_1, \dots, DLL_9, Mutex_0, Mutex_1, \dots, Mutex_9]$. Considering this, one can confidently say that DLLs and mutual exclusions are the dominant feature sets in terms of both ransomware/benign and ransomware/malware classification. Most particularly, as seen in Fig. 11b and 12b, non of the 500 API calls were more important than 10 DLLs and 10 mutual

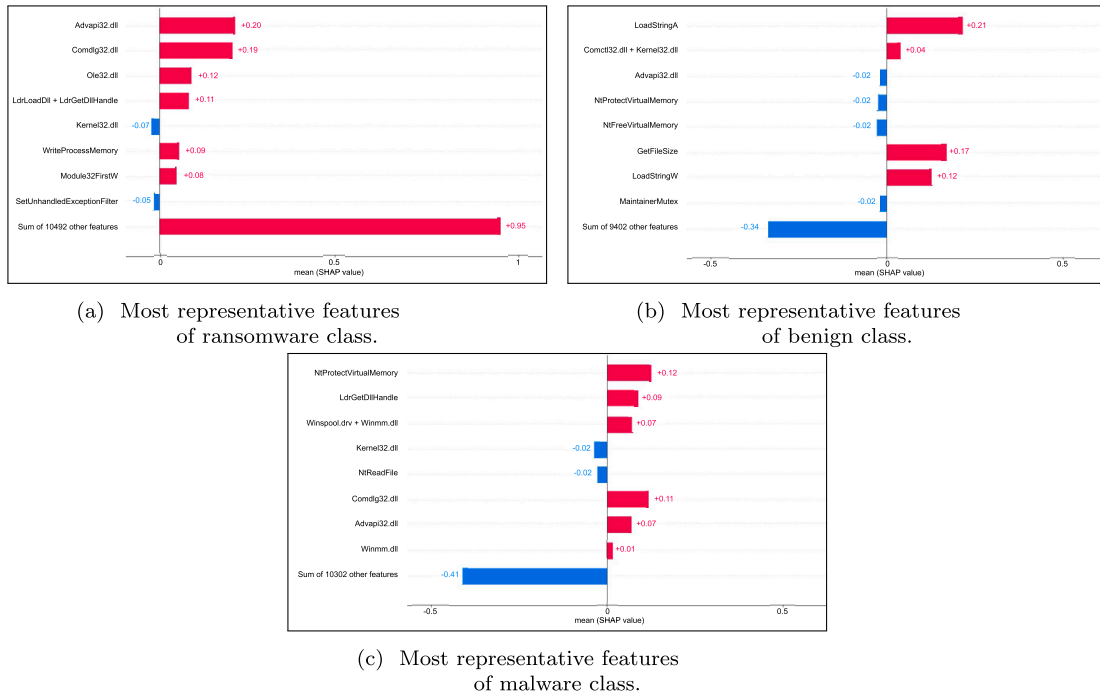


Fig. 13. Feature-based global explanation of each class.

exclusions. Such an occasion gives an idea about the RD2 dataset itself. As represented in the test results, ransomware samples from RD2 seem to be more easily detected. Our analysis on the datasets' properties reveals that RD2 has approximately half long API call sequences and half unique API calls within these sequences compared to the other datasets. Hence, the samples from RD2 have a distinctive structure than the others which facilitates the classification task.

Alongside the special case of RD2, despite being one of the most frequently used dynamic features, API calls seem to have less effect on ransomware/benign detection. However, as seen in Fig. 12, API calls become relatively more notable in terms of detecting ransomware among other malwares. With this, we can deduce that using only API calls might be enough to obtain a decent performance on malware/benign classifications, but different perspectives must be taken into consideration for ransomware/benign classification. In addition, despite having a length of 500, non of them after the first 21 seem to be crucial in terms of the classification, ignoring the special case in Fig. 12a. This knowledge indicates that the ransomware samples are prone to call the most distinctive API calls as soon as they are being executed.

Fig. 13 visualizes the particular features that have the greatest impact on representing a class: ransomware, benign, or malware. As seen, these features correspond to the inferences by interpreting the local explanations provided by LIME in Section 4.4.1. *Advapi32.dll* seems to be one of the crucial features to represent a ransomware as seen in Fig. 13a. *Ole32.dll*, which is an OLE library just as *Oleaut32.dll* also a significant feature for a ransomware. The combination of *LdrLoadDll* and *LdrGetDllHandle* demonstrates the ransomware's likelihood to load and use an external library. This is an expected behavior from a ransomware, as it is inclined to use libraries and bypass the static analysis if possible.

In Section 4.4.1, *Comctl32.dll* was seen as a ransomware-related feature for the local explanation of a sample. However, the usage of *Comctl32.dll* in conjunction with *Kernel32.dll* reflects the behavior of a benign software as seen in Fig. 13b. With this, the correlation between features rises to the occasion. Similarly, Fig. 13c demonstrates that *Winspool.drv* and *Winmm.dll* seem to be malware indicators when they are being used together. *Winspool.drv* is a library known to be related to printer drivers, while *Winmm.dll* is used to interact with multimedia devices. Thus, it can be said that both of these libraries provide functions

to perform some activities that can be considered benign-like. However, the combination of these two might signify the spreading attempt of a malware by using external hardware.

Additionally, Fig. 13c supports the previously specified circumstance of benign-coded features being correlated with malware samples in a ransomware/malware classification model. As seen, *NtProtectVirtualMemory* and *Advapi32.dll* are one of the most representative features of malware class. However, in the local explanations in Section 4.4.1, we have seen that both *Advapi32.dll* and memory-related API calls are linked with ransomware instead of malware. Also, Fig. 13c indicates that *Kernel32.dll* has a negative effect on representing a malware, but the very same feature was interpreted as a malware indicator in the local explanations provided by LIME.

5. Conclusions and future works

In this work, we propose a new XAI-empowered ransomware detection method using deep learning and dynamic analysis. Our method generates concatenated sequences that consist of multiple different sequences, such as API call, DLL, and Mutex, each representing a different view of the executable, to reveal the correlation between features belonging to different perspectives. XRun performs a 2-layered CNN that learns a joint space by capturing the correlation between API call and DLL, API call and mutex, and DLL and mutex attributes. XRun achieves 99.4% TPR for ransomware detection. Furthermore, it is similarly capable of successfully separating ransomware samples from common malware with a TPR of 94.7%. We provided a detailed local and global analysis of the patterns that distinguish ransomware from benign software using two XAI methods, such as LIME and SHAP.

As future work, we propose investigating the following possible directions. The first direction might be to extend the model to include static analysis-generated attributes such as PE header, byte data, opcode sequence. Since dynamic analysis requires using a remarkable amount of hardware resources, we will be enhancing XRun with the addition of a static analysis-based detection stage right before the dynamic analysis with the purpose of overcoming this issue. Thus, we plan to reduce time and resource consumption by applying the dynamic analysis process only for samples that cannot pass a certain confidence score in

static analysis. The second direction could be to develop a hierarchical network design that provides multi-label classification to separate ransomware types after ransomware is detected. The third direction might be developing a multi-class classification model for family identification of ransomware. Although it is a challenging task due to the evolving nature of the attackers' code-writing, such a diagnosis could be helpful in terms of understanding the aim of the ransomware more specifically.

CRedit authorship contribution statement

Sibel Gulmez: Data curation, Formal analysis, Investigation, Methodology, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Arzu Gorgulu Kakisim:** Conceptualization, Methodology, Resources, Supervision, Visualization, Writing – original draft, Writing – review & editing. **Ibrahim Sogukpinar:** Conceptualization, Funding acquisition, Project administration, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by Scientific Research Projects of Gebze Technical University, Grant No: 2022-A-113-03.

References

- Adamu, U., Awan, I., 2019. Ransomware prediction using supervised learning algorithms. In: 2019 7th International Conference on Future Internet of Things and Cloud (Fi-Cloud). IEEE, pp. 57–63.
- Alani, M.M., Mashatan, A., Miri, A., 2023. Xmal: a lightweight memory-based explainable obfuscated-malware detector. *Comput. Secur.*, 103409.
- Alhawi, O.M., Baldwin, J., Dehghantanha, A., 2018. Leveraging machine learning techniques for windows ransomware network traffic detection. In: *Cyber Threat Intelligence*, pp. 93–106.
- Almashhadani, A.O., Kaiiali, M., Sezer, S., O'Kane, P., 2019. A multi-classifier network-based crypto ransomware detection system: a case study of locky ransomware. *IEEE Access* 7, 47053–47067.
- Andronio, N., Zanero, S., Maggi, F., 2015. Heldroid: dissecting and detecting mobile ransomware. In: *Research in Attacks, Intrusions, and Defenses: 18th International Symposium. RAID 2015, Kyoto, Japan, November 2–4, 2015, Proceedings 18*. Springer, pp. 382–404.
- Arp, D., Spreitzerbarth, M., Hubner, M., Gascon, H., Rieck, K., Siemens, C., 2014. Drebin: effective and explainable detection of Android malware in your pocket. In: *Ndss*, vol. 14, pp. 23–26.
- Beaman, C., Barkworth, A., Akande, T.D., Hakak, S., Khan, M.K., 2021. Ransomware: recent advances, analysis, challenges and future research directions. *Comput. Secur.* 111, 102490.
- BlackFog, 2023. The state of ransomware in 2023. <https://www.blackfog.com/the-state-of-ransomware-in-2023/>. (Accessed 8 February 2023).
- Chen, Z.-G., Kang, H.-S., Yin, S.-N., Kim, S.-R., 2017. Automatic ransomware detection and analysis based on dynamic api calls flow graph. In: *Proceedings of the International Conference on Research in Adaptive and Convergent Systems*, pp. 196–201.
- CNET, 2023. Free software downloads. <https://download.cnet.com>. (Accessed 8 July 2023).
- Conti, M., Khandhar, S., Vinod, P., 2022. A few-shot malware classification approach for unknown family recognition using malware feature visualization. *Comput. Secur.* 122, 102887.
- Continella, A., Guagnelli, A., Zingaro, G., De Pasquale, G., Barengi, A., Zanero, S., Maggi, F., 2016. Shields: a self-healing, ransomware-aware filesystem. In: *Proceedings of the 32nd Annual Conference on Computer Security Applications*, pp. 336–347.
- Cuckoo Sandbox, 2019. Cuckoo Sandbox - Automated Malware Analysis. <https://cuckoosandbox.org>. (Accessed 8 May 2023).
- Fan, M., Wei, W., Xie, X., Liu, Y., Guan, X., Liu, T., 2020. Can we trust your explanations? Sanity checks for interpreters in Android malware analysis. *IEEE Trans. Inf. Forensics Secur.* 16, 838–853.
- Feichtner, J., Gruber, S., 2020. Understanding privacy awareness in Android app descriptions using deep learning. In: *Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy*, pp. 203–214.
- Free Automated Malware Analysis Service, 2020. <https://hybrid-analysis.com>. (Accessed 27 October 2023).
- FreewareFiles, 2022. <https://www.freewarefiles.com>. (Accessed 27 October 2023).
- Guidotti, R., Monreale, A., Ruggieri, S., Pedreschi, D., Turini, F., Giannotti, F., 2018. Local rule-based explanations of black box decision systems. *arXiv preprint. arXiv:1805.10820*.
- Gulmez, S., Kakisim, A.G., Sogukpinar, I., 2023. Analysis of the dynamic features on ransomware detection using deep learning-based methods. In: *2023 11th International Symposium on Digital Forensics and Security (ISDFS)*. IEEE, pp. 1–6.
- Guo, W., Mu, D., Xu, J., Su, P., Wang, G., Xing, X., 2018. Lemna: explaining deep learning based security applications. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pp. 364–379.
- Harang, R., Rudd, E.M., 2020. Sorel-20m: a large scale benchmark dataset for malicious pe detection. *arXiv preprint. arXiv:2012.07634*.
- Hasan, M.M., Rahman, M.M., 2017. Ranshunt: a support vector machines based ransomware analysis framework with integrated feature set. In: *2017 20th International Conference of Computer and Information Technology (ICCIT)*. IEEE, pp. 1–7.
- Hsupeng, B., Lee, K.-W., Wei, T.-E., Wang, S.-H., 2022. Explainable malware detection using predefined network flow. In: *2022 24th International Conference on Advanced Communication Technology (ICACT)*. IEEE, pp. 27–33.
- Hwang, J., Kim, J., Lee, S., Kim, K., 2020. Two-stage ransomware detection using dynamic analysis and machine learning techniques. *Wirel. Pers. Commun.* 112, 2597–2609.
- Jackson, Don, 2007. Threat analysis - Gozi trojan. <https://www.secureworks.com/research/gozi>. (Accessed 15 August 2023).
- Jethva, B., Traoré, I., Ghaleb, A., Ganame, K., Ahmed, S., 2020. Multilayer ransomware detection using grouped registry key operations, file entropy and file signature monitoring. *J. Comput. Secur.* 28 (3), 337–373.
- Karbab, E.B., Debbabi, M., Derhab, A., 2023. Swift: cross-platform ransomware fingerprinting using hierarchical neural networks on hybrid features. *Expert Syst. Appl.* 225, 120017.
- Keshk, M., Koroniotis, N., Pham, N., Moustafa, N., Turnbull, B., Zomaya, A.Y., 2023. An explainable deep learning-enabled intrusion detection framework in iot networks. *Inf. Sci.* 639, 119000.
- Kharaz, A., Arshad, S., Mulliner, C., Robertson, W., Kirda, E., 2016. (UNVEIL): a (Large-Scale), automated approach to detecting ransomware. In: *25th USENIX Security Symposium (USENIX Security 16)*, pp. 757–772.
- Kinlead, M., Millar, S., McLaughlin, N., O'Kane, P., 2021. Towards explainable cnns for Android malware detection. *Proc. Comput. Sci.* 184, 959–965.
- Lee, K., Lee, S.-Y., Yim, K., 2019. Machine learning based file entropy analysis for ransomware detection in backup systems. *IEEE Access* 7, 110205–110215.
- Lundberg, S.M., Lee, S.-I., 2017. A unified approach to interpreting model predictions. *Adv. Neural Inf. Process. Syst.* 30.
- Manavi, F., Hamzeh, A., 2022a. A novel approach for ransomware detection based on pe header using graph embedding. *J. Comput. Virol. Hacking Tech.* 18 (4), 285–296.
- Manavi, F., Hamzeh, A., 2022b. Ransomware detection based on pe header using convolutional neural networks. *ISeCure* 14 (2).
- McIntosh, T., Kayes, A., Chen, Y.-P.P., Ng, A., Watters, P., 2021. Dynamic user-centric access control for detection of ransomware attacks. *Comput. Secur.* 111, 102461.
- Morato, D., Berrueta, E., Magaña, E., Izal, M., 2018. Ransomware early detection by the analysis of file sharing traffic. *J. Netw. Comput. Appl.* 124, 14–32.
- Moreira, C.C., Moreira, D.C., de Sales, C.d.S. Jr., 2023. Improving ransomware detection based on portable executable header using xception convolutional neural network. *Comput. Secur.* 130, 103265.
- Pan, Z., Sheldon, J., Mishra, P., 2022. Hardware-assisted malware detection and localization using explainable machine learning. *IEEE Trans. Comput.* 71 (12), 3308–3321.
- Portable Freeware, 2023. <https://www.portablefreeware.com>. (Accessed 27 October 2023).
- Qin, B., Wang, Y., Ma, C., 2020. Api call based ransomware dynamic detection approach using textcnn. In: *2020 International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)*. IEEE, pp. 162–166.
- Ribeiro, M.T., Singh, S., Guestrin, C., 2016. “Why should I trust you?” explaining the predictions of any classifier. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144.
- Ribeiro, M.T., Singh, S., Guestrin, C., 2018. Anchors: high-precision model-agnostic explanations. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32.
- Sangfor Technologies, 2020. Cyber security — enterprise cloud — Sangfor technologies. <https://www.sangfor.com>. (Accessed 27 October 2023).
- Sgandurra, D., Muñoz-González, L., Mohsen, R., Lupu, E.C., 2016. Automated dynamic analysis of ransomware: benefits, limitations and use for detection. *arXiv preprint. arXiv:1609.03020*.
- Shaukat, S.K., Ribeiro, V.J., 2018. Ransomwall: a layered defense system against cryptographic ransomware attacks using machine learning. In: *2018 10th International Conference on Communication Systems & Networks (COMSNETS)*. IEEE, pp. 356–363.
- Sheen, S., Yadav, A., 2018. Ransomware detection by mining api call usage. In: *2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*. IEEE, pp. 983–987.

- Shibahara, T., Yagi, T., Akiyama, M., Chiba, D., Yada, T., 2016. Efficient dynamic malware analysis based on network behavior using deep learning. In: 2016 IEEE Global Communications Conference (GLOBECOM). IEEE, pp. 1–7.
- Shibahara, T., Yagi, T., Akiyama, M., Chiba, D., Hato, K., 2019. Efficient dynamic malware analysis for collecting http requests using deep learning. *IEICE Trans. Inf. Syst.* 102 (4), 725–736.
- SnapFiles, 2022. SnapFiles freeware and shareware downloads and reviews. <https://www.snapfiles.com>. (Accessed 27 October 2023).
- Softonic, 2020. <https://en.softonic.com>. (Accessed 27 October 2023).
- Software Informer, 2016. Software Informer - windows software downloads and editorial reviews. <https://software.informer.com>. (Accessed 27 October 2023).
- SonicWall, 2023. 2023 SonicWall cyber threat report. <https://www.sonicwall.com/2023-cyber-threat-report/>. (Accessed 31 July 2023).
- SonicWALL Security Center, 2020. <http://software.sonicwall.com/applications/gav/index.asp?ev=sig&sigid=12391>. (Accessed 15 August 2023).
- Statista, 2023. Percentage of organizations victimized by ransomware attacks worldwide from 2018 to 2022. <https://www.statista.com/statistics/204457/businesses-ransomware-attack-rate/>. (Accessed 31 July 2023).
- Suhail, S., Iqbal, M., Hussain, R., Jurdak, R., 2023. Enigma: an explainable digital twin security solution for cyber-physical systems. *Comput. Ind.* 151, 103961.
- VirusShare, 2016. <https://www.virusshare.com>. (Accessed 8 July 2023).
- VirusTotal, 2018. <https://www.virustotal.com>. (Accessed 27 October 2023).
- VX Heavens, 2010. <https://archive.org/download/vxheavens-2010-05-18>. (Accessed 8 August 2023).
- Wu, B., Chen, S., Gao, C., Fan, L., Liu, Y., Wen, W., Lyu, M.R., 2021. Why an Android app is classified as malware: toward malware classification interpretation. *ACM Trans. Softw. Eng. Methodol.* 30 (2), 1–29.
- Wu, Y., Zhang, B., Lai, Z., Su, J., 2012. Malware network behavior extraction based on dynamic binary analysis. In: 2012 IEEE International Conference on Computer Science and Automation Engineering. IEEE, pp. 316–320.

- Yang, W., Gao, M., Chen, L., Liu, Z., Ying, L., 2023. Recmal: rectify the malware family label via hybrid analysis. *Comput. Secur.* 128, 103177.



Sibel Gulmez is pursuing her Ph.D. at the Department of Computer Engineering, Gebze Technical University. Her research area includes analysis and the detection/classification of malware. Currently, she is working as a Research Assistant at Gebze Technical University.



Arzu Gorgulu Kakisim is currently an Assistant Professor at the Department of Computer Engineering, Istanbul Medeniyet University, Turkey. She received her Ph.D. degree from the Department of Computer Engineering, Gebze Technical University, Turkey, in 2019. Her primary research interests are text and graph mining, representation learning for heterogeneous networks, and malware detection.



Ibrahim Sogukpinar received his B.Sc. degree in Electronics and Communications Engineering from the Technical University of Istanbul, his M.Sc. and Ph.D. degrees in Computer and Control Engineering from the Technical University of Istanbul. Currently, he is the Professor at Computer Engineering Department at Gebze Technical University. His interested areas are information/cyber security, computer networks, applications of information systems.